

# **INSTITUTO POLITÉCNICO NACIONAL**

## **Escuela Superior de Cómputo**

**Unidad de Aprendizaje:** Bases de Datos  
**Semestre:** 4to Semestre

### **Diseño, Implementación y Operaciones de Base de Datos**

## **“Refaccionaria Leo”**

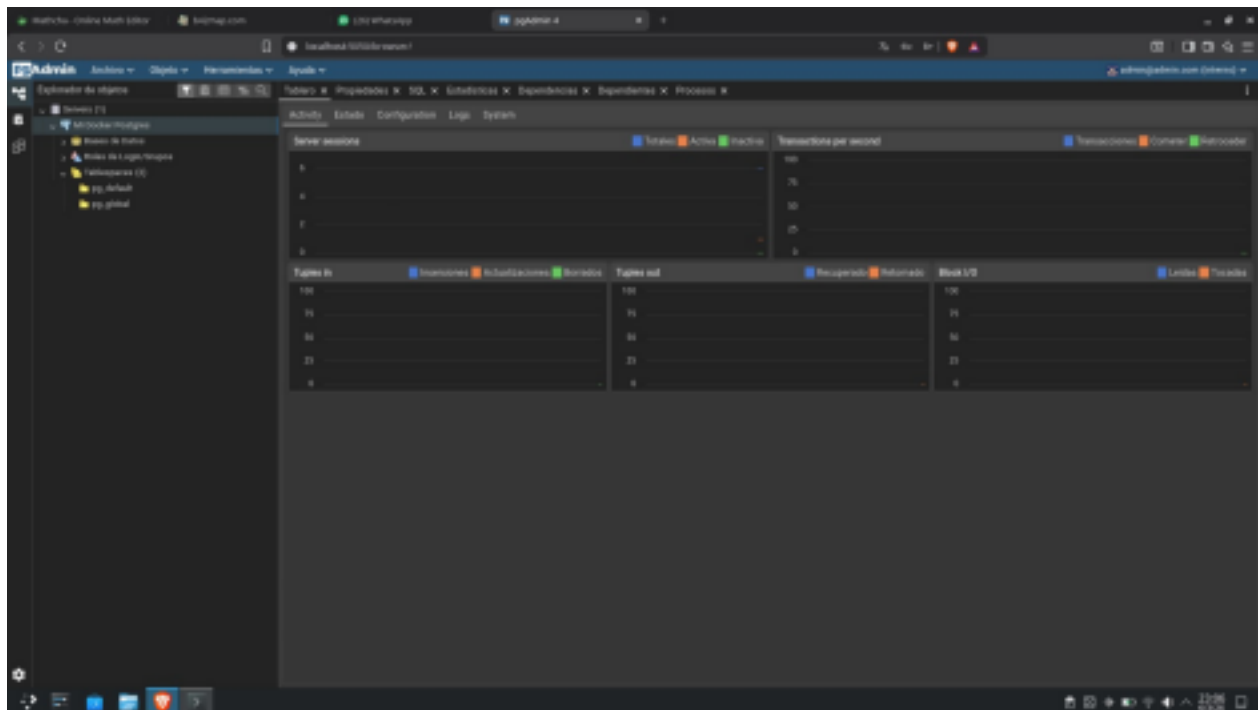
#### **Contenido del Documento:**

- Transformación del Modelo Entidad-Relación (EER) al Relacional
- Diccionario de Datos y Creación de Esquema Físico (DDL) en PostgreSQL
- Estrategias de Poblado de Datos y Gestión de Usuarios (DCL)
- Operaciones DML Avanzadas y Cuestionario de Integridad Referencial

#### **Integrantes del Equipo:**

Martínez Marín Nahum  
Miranda Arredondo Miguel Angel

Ciudad de México, a 8 de junio de 2026

[illegible]

## 2.1. Sobre el Inventario y Catálogo

Pregunta: Para la compatibilidad de autos, ¿qué tan específicos debemos ser?

Respuesta: Específico hasta el nivel de Motor. La tabla Vehículo tiene los atributos Marca, Modelo, Año y Motor. Si la refaccionaria necesita filtrar por "Transmisión" (Automática/Manual) o "Versión" (Lujo/Austero), el modelo actual fallaría y requeriría un ALTER TABLE para agregar esas columnas.

Pregunta: ¿Qué debe pasar cuando un producto llega a 0 en stock?

Respuesta: Se delega al código. La tabla Producto tiene el atributo Stock (INT). Sin embargo, no existe un campo como Activo o Visible (Booleano). La base de datos no lo oculta automáticamente; el backend de la página web tendría que programar la lógica IF Stock == 0 para mostrar el letrero de "Agotado".

## 2.2. Sobre los Clientes y las Ventas

Pregunta: ¿Le vendes solo a clientes finales o también a talleres mecánicos con precio especial?

Respuesta: Trata a todos por igual. La tabla Cliente asume un solo tipo de usuario final. No existe un atributo de Rol ni una tabla de Niveles Descuento. Para darle precio de mayoreo a un taller, tendríamos que reestructurar los atributos del cliente.

Pregunta: ¿Manejan créditos o pagos a plazos (cuentas por cobrar a talleres)?

Respuesta: Diseñado para pago inmediato. La tabla Pedido tiene Método Pago y Estado Pedido, pero no hay estructura para registrar abonos, fechas de vencimiento de crédito o saldos pendientes.

Pregunta: ¿Los precios de los productos ya incluyen IVA?

Respuesta: Precio unificado. En Producto solo existe Precio Actual y en Pedido solo está el Total. No hay campos a nivel de base de datos para separar el Subtotal de los Impuestos. Esto asume que el backend hace el cálculo matemático en tiempo real, o que los precios ya tienen el impuesto incluido.

## 2.3. Sobre la Logística y Envíos a Domicilio

Pregunta: ¿Cómo funciona tu entrega a domicilio (repartidores propios o paquetería)? Respuesta: Soporta ambos (Flexible). La tabla Envío cuenta con Repartidor Paquetería (donde puedes escribir "Juan en Moto." "FedEx") y un Número Guía opcional, cubriendo ambos escenarios sin problema.

Pregunta: ¿Cómo se calcula el costo del envío?

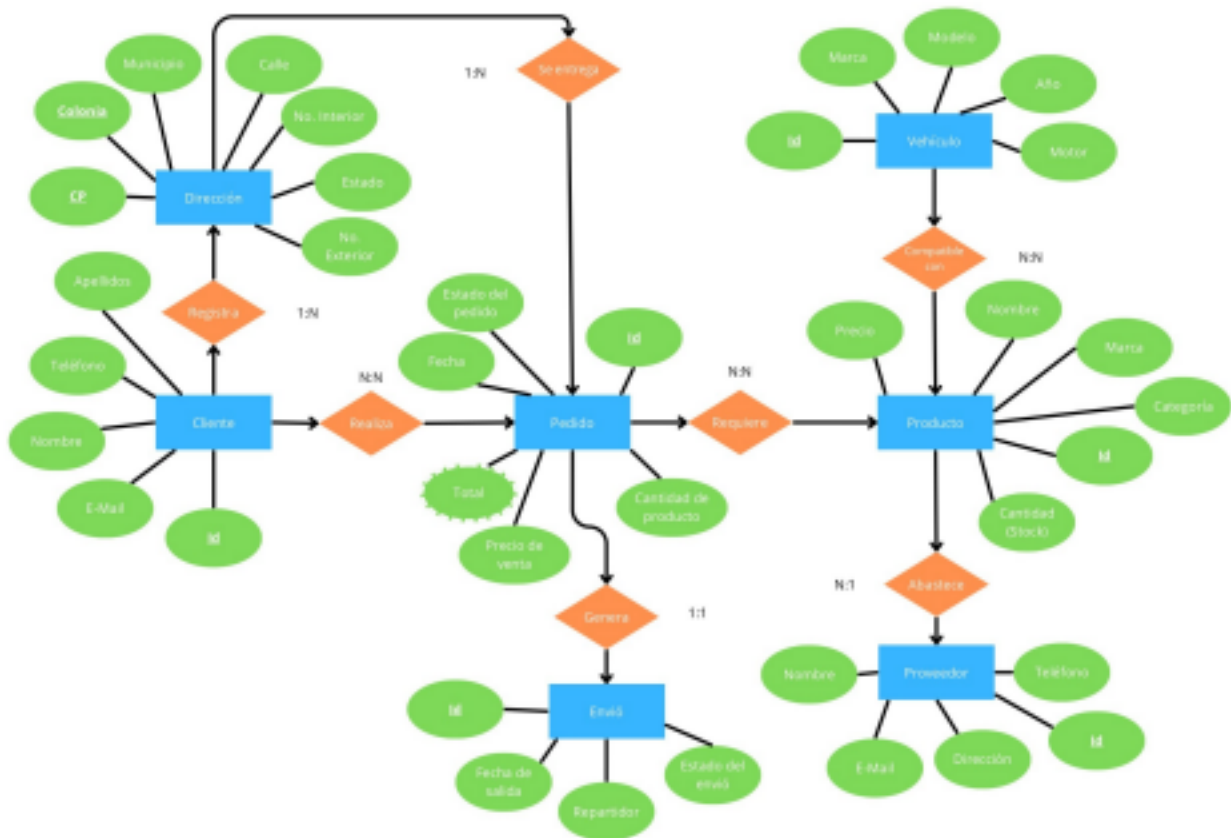
Respuesta: Se asigna manualmente. La tabla Pedido incluye el atributo Costo Envío con valor por defecto de 0.00. No hay tablas de "Zonas Tarifarias" ni cálculo por peso. El código web tendría que calcular el costo y enviarle el número final a este campo en la base de datos.

## 2.4. Sobre Seguridad y Pagos

Pregunta: ¿A través de qué plataformas vas a cobrar en línea?

Respuesta: Seguro por diseño. La tabla Pedido solo guarda el texto en Metodo Pago (Ej. "Tarjeta", "PayPal"). Fiel a las buenas prácticas de seguridad de la información, el modelo omite intencionalmente cualquier tabla o campo para guardar números de tarjetas de crédito o CVV, evitando almacenar datos críticos no encriptados.

### 3. DIAGRAMA ENTIDAD-RELACION



## 1.1 Revisión del Modelo de la Práctica 2

Retome el modelo EER completo desarrollado en la Práctica 2. Realice un inventario de todos los elementos que componen su modelo:

### Entidades Fuertes

Son aquellas representadas con un rectángulo simple y que poseen al menos un atributo llave (subrayado con línea continua).

- Cliente  
Clave Primaria: Id Cliente  
Atributos: Nombre, Apellidos, Teléfono, E-Mail
- Pedido  
Clave Primaria: Id Pedido  
Atributos: Fecha Pedido
- Metodo Pago  
Clave Primaria: Id Pago  
Atributos: Monto, Fecha Pago, Estado del Pago, Metodo Pago
- Detalle Pedido (*Nota: Aunque lógicamente a menudo es débil, en este diagrama está dibujada como fuerte*)  
Clave Primaria: Folio  
Atributos: Cantidad, Total, Precio Unidad
- Producto  
Clave Primaria: Id  
Atributos: Nombre, Marca, Categoría, Precio, Cantidad (Stock)
- Vehículo  
Clave Primaria: Id Vehículo  
Atributos: Marca, Modelo, Año, Motor
- Proveedor  
Clave Primaria: Id Proveedor  
Atributos: Nombre, Teléfono, E-Mail, Dirección
- Envío  
Clave Primaria: Id Envío  
Atributos: Fecha salida, Repartidor, Estado del envío

### Entidades Débiles

Son aquellas que dependen de otra entidad para existir, representadas con un doble rectángulo.

- Dirección Envío  
Atributos: Municipio, Calle, No. Interior, No. Exterior, CP, Colonia, Estado. (*En el diagrama no se le asignó visualmente un atributo discriminador/llave parcial*).

Relación de Identificación: Registra (representada con un doble rombo), que la vincula de forma existencial con la entidad fuerte Cliente.

### Relaciones

Todas son relaciones binarias. Las cardinalidades extraídas de las etiquetas de texto en el código son las siguientes:

- Registra (1:N): 1 Cliente registra N Dirección Envío. (Relación identificadora).
- Genera (1:N): 1 Cliente genera N Pedidos.
- Tiene (N:1): N Pedidos tienen 1 Metodo Pago.
- Tiene (1:N): 1 Pedido tiene N Detalle Pedido.
- Contiene (N:1): N Detalle Pedido contienen 1 Producto.
- Compatible con (N:N): N Vehículos son compatibles con N Productos.
- Abastece (1:N): 1 Proveedor abastece N Productos.
- Crea (1:1): 1 pedido En Linea crea 1 Envío.

### Jerarquías ISA (Generalización/Especialización)

El diagrama muestra una jerarquía usando el triángulo invertido rojo marcado con la palabra "Es".

- Superclase: Pedido
- Subclases: Venta Caja (Atributo: Cobro Caja), En Linea (Atributo: Costo Envío)
- Restricciones: El diagrama no especifica visualmente las restricciones lógicas (no hay letras "d.o." dentro del círculo, ni líneas dobles en la herencia). Sin embargo, por la naturaleza del negocio, se infiere que es una jerarquía Disjunta (un pedido es en caja o es en línea, no ambos) y Total (todo pedido debe pertenecer a una de las dos modalidades).

### Atributos Especiales

Analizando el código, no se identifican atributos especiales visualmente.

- Multivaluados: No hay elipses dobles.
- Compuestos: No hay atributos que se ramifiquen en otros atributos (por ejemplo, "Dirección.<sup>en</sup> Proveedor es un solo óvalo, no se dividió).
- Derivados: No hay elipses con líneas punteadas (por ejemplo, "Total.<sup>en</sup> Detalle Pedido o Cantidad (Stock).<sup>en</sup> Producto podrían ser calculados, pero el diagrama los trata como atributos base normales).

### Relaciones de Orden Superior

No existen relaciones ternarias ni de orden superior en este diagrama. Todas las conexiones (rombos) unen estrictamente a dos entidades.



## 1.2 Preparación para la Transformación

### 1. ¿Qué elementos requerirán tablas adicionales?

En el paso al modelo relacional, además de crear una tabla por cada entidad fuerte, se generarán tablas adicionales por las siguientes razones:

Relaciones Muchos a Muchos (N:N): La relación “Compatible con” requerirá obligatoriamente crear una nueva tabla intermedia (puedes llamarla Vehículo Producto o Compatibilidad). Esto es necesario para romper la cardinalidad N:N entre Vehículo y Producto.

Entidades Débiles: Dirección Envío se transformará en una tabla. Aunque lógicamente es una extensión del cliente, en la base de datos física debe existir como una tabla independiente para soportar la cardinalidad 1:N (un cliente puede tener varias direcciones registradas).

*(Nota: Si existieran atributos multivaluados como “Varios teléfonos”, requerirían otra tabla, pero en tu diagrama no hay ninguno).*

### 2. ¿Qué atributos se convertirán en claves foráneas (Foreign Keys)?

Las claves foráneas (FK) nacen al absorber las relaciones 1:N y 1:1. La regla general es que la tabla que tiene la cardinalidad “N” recibe la clave primaria de la tabla del lado “1”.

En Dirección Envío: Recibirá Id Cliente. (Ojo: al ser entidad débil, esta FK formará parte de su propia Clave Primaria compuesta).

En Pedido: Recibirá Id Cliente (del cliente que lo Genera) y recibirá Id Pago (del Método de Pago que Tiene).

En Detalle Pedido: Recibirá Id Pedido (para saber a qué compra pertenece) y Id Producto (para saber qué artículo contiene).

En Producto: Recibirá Id Proveedor (del proveedor que lo Abastece).

En la nueva tabla Compatible Con: Recibirá Id Vehículo e Id Producto (ambas serán FK y juntas formarán la PK de esta tabla).

En Envío: Recibirá la clave de la venta en línea (ej. Id Pedido) debido a la relación “Crea” (1:1).

### 3. ¿Qué decisiones de diseño deberá tomar en las jerarquías de herencia?

Para la herencia donde Pedido es la superclase y Venta Caja / En Línea son las sub clases, tendrás que tomar una decisión de diseño arquitectónico. Tienes 3 opciones (estrategias de mapeo), y debes elegir una:

Opción A (1 Tabla Única - “Single Table”): Creas una sola tabla llamada Pedido. Contendrá los atributos base (Id Pedido, Fecha Pedido) y también

los específicos (Cobro Caja, Costo Envío).

- ¿Qué debes añadir? Un atributo discriminador extra (ej. Tipo Pedido que diga “Caja” o “En Linea”).

3

- Pro/Contra: Es la más rápida para hacer consultas, pero tendrás muchos campos con valores “NULL” (ej. si es venta en caja, Costo Envío estaría vacío).

Opción B (3 Tablas - “Class Table”): Creas una tabla para Pedido, otra para Venta Caja y otra para En Linea.

- ¿Cómo se conectan? Venta Caja y En Linea usarán Id Pedido como su Clave Primaria y también como Clave Foránea apuntando a la tabla principal Pedido.
- Pro/Contra: Es la más limpia y fiel al diagrama, no hay nulos, pero requeriría usar “JOINS” en SQL cada vez que quieras ver la información completa de una venta.

Opción C (2 Tablas concretas - “Concrete Table”): Eliminas la tabla principal Pedido. Creas solo Venta Caja y En Linea y a ambas les repites los atributos de la superclase (es decir, ambas tendrán Id Pedido y Fecha Pedido).

- Pro/Contra: Ideal si los procesos de caja y en línea son en sistemas completamente separados, pero mala idea si necesitas reportes globales de todas las ventas juntas.

## 2.1 Transformación de Entidades Fuertes

Tabla: Cliente

Id Cliente (PK): INT [increment]

Nombre: VARCHAR(100) [not null]

Apellidos: VARCHAR(100) [not null]

Telefono: VARCHAR(15)

Email: VARCHAR(150)

Tabla: Proveedor

Id Proveedor (PK): INT [increment]

Nombre: VARCHAR(100) [not null]

Telefono: VARCHAR(15)

Email: VARCHAR(150)

Direccion: TEXT

Tabla: Vehiculo

Id Vehiculo (PK): INT [increment]

4

Marca: VARCHAR(50) [not null]

Modelo: VARCHAR(50) [not null]

Anio: INT

Motor: VARCHAR(50)

Tabla: Metodo Pago

Id Pago (PK): INT [increment]

Monto: DECIMAL(10,2) [not null]

Fecha Pago: DATE [not null]

Estado Pago: VARCHAR(50) [not null]

Tipo Metodo: VARCHAR(50) [not null]

Tabla: Producto

Id Producto (PK): INT [increment]

Nombre: VARCHAR(100) [not null]

Marca: VARCHAR(50)

Categoria: VARCHAR(50)

Precio: DECIMAL(10,2) [not null]

Stock: INT [not null, default: 0]

Id Proveedor (FK): INT

Tabla: Pedido

Id Pedido (PK): INT [increment]

Fecha Pedido: DATE [not null, default: CURRENT  
DATE] Id Cliente (FK): INT [not null]

Id Pago (FK): INT [not null]

Tabla: Detalle Pedido

Folio (PK): INT [increment]

Cantidad: INT [not null]

Precio Unidad: DECIMAL(10,2) [not null]

5

Total: DECIMAL(10,2) [not null]

Id Pedido (FK): INT [not null]

Id Producto (FK): INT [not null]

Tabla: Envio

Id Envio (PK): INT [increment]

Fecha Salida: DATE

Repartidor: VARCHAR(100)

Estado Envio: VARCHAR(50) [not null, default: 'En preparaci3n']

Id Pedido En Linea (FK): INT [not null, unique]

## 2.2 Transformaci3n de Atributos Compuestos

Resultado de la evaluaci3n: No se identificaron atributos compuestos en el modelo conceptual de origen. Todos los atributos documentados poseen una estructura at3mica simple. Por consiguiente, no se requiere la ejecuci3n de procesos de descomposici3n estructural ni la creaci3n de columnas adicionales bajo esta regla.

## 2.3 Transformaci3n de Atributos Multivaluados

Resultado de la evaluaci3n: El modelo conceptual no presenta atributos de naturaleza multivaluada en ninguna de sus entidades. Debido a la ausencia de listas de valores m3ltiples incrustadas en campos individuales, no procede la generaci3n de tablas independientes ni la conformaci3n de claves primarias compuestas derivadas de esta regla.

## 3.1 Relaciones Uno a Muchos (1:N)

Resultado de la evaluaci3n: Aplicando la Regla 4, se identificaron m3ltiples relaciones 1:N en el modelo. La clave primaria del lado “uno” se propag3o como clave for3nea (FK) al lado “muchos”.

Relación: Cliente (1) → Dirección Envío (N)

- Lado Uno: Cliente
- Lado Muchos: Dirección Envío
  - Propagación: Id Cliente se integra a la tabla DIRECCION

## ENVIO. 6

- Restricción: NOT NULL. Al ser una entidad débil, la existencia de la dirección depende de la existencia de un cliente. (Además, forma parte de la clave primaria compuesta).

Relación: Proveedor (1) → Producto (N)

- Lado Uno: Proveedor
- Lado Muchos: Producto
- Propagación: Id Proveedor se integra a la tabla PRODUCTO.
- Restricción: Acepta NULL (implícito en el código, ya que no se declaró explícitamente [not null]). Esto indica una cardinalidad mínima (0,N), sugiriendo que un producto puede estar registrado sin un proveedor asignado temporalmente.

Relación: Cliente (1) → Pedido (N)

- Lado Uno: Cliente
- Lado Muchos: Pedido
- Propagación: Id Cliente se integra a la tabla PEDIDO.
- Restricción: NOT NULL. Todo pedido debe pertenecer a un cliente.

Relación: Metodo Pago (1) → Pedido (N)

- Lado Uno: Metodo Pago
- Lado Muchos: Pedido
- Propagación: Id Pago se integra a la tabla PEDIDO.
- Restricción: NOT NULL. Todo pedido requiere un método de pago

registrado. Relación: Pedido (1) → Detalle Pedido (N)

- Lado Uno: Pedido
- Lado Muchos: Detalle Pedido

- Propagación: Id Pedido se integra a la tabla DETALLE PEDIDO. •
- Restricción: NOT NULL. Todo detalle debe pertenecer a una orden válida.

Relación: Producto (1) → Detalle Pedido (N)

- Lado Uno: Producto
- Lado Muchos: Detalle Pedido
- Propagación: Id Producto se integra a la tabla DETALLE PEDIDO. •
- Restricción: NOT NULL. Todo detalle debe especificar qué producto contiene.

## 7

### 3.2 Relaciones Muchos a Muchos (N:M)

Resultado de la evaluación: Aplicando la Regla 5, se identificó una relación N:M que requiere una tabla asociativa.

Relación: Producto (N) ↔ Vehículo (M) (Relación “Compatible con”)

- Acción: Creación de la tabla asociativa PRODUCTO VEHICULO.
- Claves Foráneas: Se incluyen Id Producto (de la tabla PRODUCTO) e Id Vehiculo (de la tabla VEHICULO).
- Clave Primaria: La combinación de ambas claves foráneas: (Id Producto, Id Vehiculo) [pk].
- Atributos propios: No se definieron atributos adicionales para esta relación en el diseño actual.

### 3.3 Relaciones Uno a Uno (1:1)

Resultado de la evaluación: Aplicando la Regla 6, se identificaron relaciones 1:1, derivadas principalmente de procesos de herencia y logística. Se ha empleado la estrategia de propagación con restricciones de unicidad o coincidencia de PK.

Relación: Pedido (1) → Venta Caja (1) (Jerarquía de Herencia - Subtipo)

- Estrategia Aplicada: Se utilizó una variación de la Opción B/C (Mapeo de Clases - “Class Table”). La tabla subtipo VENTA CAJA utiliza Id Pedido tanto como Clave Primaria (PK) propia, como Clave Foránea (FK) hacia PEDIDO. Esto garantiza una relación estrictamente 1:1.
- Justificación: Mantiene la integridad referencial de la herencia sin usar atributos nulos masivos en la tabla principal.

Relación: Pedido (1) → En Linea (1) (Jerarquía de Herencia - Subtipo)

- Estrategia Aplicada: Idéntica al caso anterior. La tabla EN LINEA usa Id Pedido como PK y FK concurrente hacia PEDIDO.
- Justificación: Implementación estándar para mapear jerarquías disjuntas a nivel relacional.

Relación: En Linea (1) → Envío (1) (Relación "Crea")

- Estrategia Aplicada: Opción A. Propagación de clave con restricción de unicidad.
- Implementación: La clave primaria Id Pedido de la tabla EN LINEA se propaga hacia la tabla ENVIO como la columna Id Pedido En Linea.
- Restricción: Se le aplica explícitamente el constreñimiento [unique] y [not

null]. 8

- Justificación: Garantiza matemáticamente que un pedido en línea específico no pueda generar más de un registro de envío, satisfaciendo la cardinalidad 1:1 requerida por las reglas de negocio de la refaccionaria.

### 3.4 Relaciones Ternarias y de Orden Superior

Resultado de la evaluación: Tras el escrutinio del modelo y del código SQL proporcionado, no se identificaron relaciones de orden superior (ternarias, cuaternarias, etc.). Todas las interacciones modeladas entre las entidades operan bajo arquitecturas estrictamente binarias. En consecuencia, la Regla 7 no es aplicable a esta versión de la base de datos.

### 4.1 Entidades Débiles con Dependencia de Identificación

Resultado de la evaluación: Aplicando la Regla 8, se identificó una entidad débil por identificación en el modelo, la cual fue transformada siguiendo los estándares de claves compuestas y restricciones de integridad.

#### 1. Entidad Débil: Dirección Envío

Entidad Propietaria (Fuerte): Cliente

Transformación: Se creó la tabla DIRECCION ENVIO con sus atributos propios (Municipio, Calle, No Interior, No Exterior, CP, Colonia, Estado).

Propagación de Clave: Se agregó Id Cliente (PK de la entidad propietaria) como Clave Foránea (FK).

Clave Primaria Compuesta: La PK de la tabla se conformó por la unión de la FK y un atributo discriminador: Indexes { (Id Cliente, Numero Consecutivo) [pk]

} El atributo Numero Consecutivo actúa como el discriminador (ej. Dirección 1, Dirección 2 del mismo cliente).

Restricciones de Integridad:

- La FK se definió como obligatoria: Id Cliente INT [not null].
- Se configuró la eliminación en cascada para garantizar la dependencia existencial: Ref: DIRECCION ENVIO.Id Cliente >CLIENTE.Id Cliente [delete: cascade]. Si se elimina un cliente, sus direcciones desaparecen automáticamente.

## 4.2 Entidades Débiles con Dependencia de Existencia

Resultado de la evaluación: Se identificaron entidades que poseen un identificador propio (llave subrogada), pero cuya existencia lógica y física depende absolutamente de otra entidad principal.

### 1. Entidad Débil por Existencia: Detalle Pedido

9

Entidad Propietaria: Pedido

Transformación: Aunque lógicamente es un detalle dependiente, se le asignó su propia Clave Primaria independiente: Folio INT [pk, increment].

Restricciones aplicadas:

- Se incluyó la FK hacia la propietaria de forma obligatoria: Id Pedido INT [not null].
- Se configuró la restricción de borrado: Ref: DETALLE PEDIDO.Id Pedido >PEDIDO.Id Pedi[delete: cascade]. Esto asegura que si se cancela o elimina un pedido principal, todas sus líneas de detalle (productos comprados) sean eliminadas para no dejar datos huérfanos.

### 2. Entidad Débil por Existencia: Envío

Entidad Propietaria: En Línea (Subtipo de Pedido)

Transformación: Se creó la tabla ENVIO con su propio identificador único: Id Envío INT [pk, increment]. Un envío no tiene sentido sin una compra en línea que lo origine.

Restricciones aplicadas:

- Se incluyó la FK obligatoria: Id Pedido En Línea INT [not null].
- Se aplicó la eliminación en cascada: Ref: ENVIO.Id Pedido En Línea - EN LINEA.Id Pedi[delete: cascade].

## 5.1 y 5.2 Aplicación de Estrategias de Transformación a Jerarquías ISA

Análisis de la Jerarquía Identificada

Supertipo: PEDIDO



Subtipos: VENTA CAJA y EN LINEA

### 1. Análisis de Restricciones Lógicas

¿Es disjunta o solapada? Es lógicamente Disjunta. Un pedido se realiza presencialmente en la sucursal (caja) o se realiza a través de la plataforma web (en línea). No puede ser ambas cosas simultáneamente bajo un mismo folio.

¿Es total o parcial? Es lógicamente Total. Todo pedido registrado en el sistema debe categorizarse forzosamente bajo una de estas dos modalidades; no existen pedidos “genéricos” que no sean ni en caja ni en línea.

### 2. Elección y Justificación de la Estrategia

Estrategia Seleccionada: Se implementó la Estrategia A: Múltiples Tablas con Relaciones 1:1 (reflejado en el código de dbdiagram.io proporcionado).

Justificación:

10

Integridad y Normalización: La Estrategia A permite mantener una tabla central (PEDIDO) con el historial global de ventas (Fecha Pedido, Id Cliente, Id Pago), lo cual es crucial para reportes financieros generales (Ej. “Ventas totales del mes”), independientemente del canal de origen.

Manejo de Dependencias Complejas: Si hubiésemos elegido la Estrategia C (eliminar la tabla PEDIDO), habríamos tenido que duplicar la relación con DETALLE PEDIDO (apuntando a veces a caja y a veces en línea), lo cual fragmentaría severamente la integridad del carrito de compras.

Evitar Nulos (Contra Estrategia B): Los pedidos EN LINEA desencadenan un proceso logístico completo (la entidad ENVIO), mientras que VENTA CAJA no. Si usáramos una sola tabla, tendríamos campos de envío y costos de paquetería permanentemente vacíos en todas las compras físicas, desperdiciando espacio y ensuciando la estructura de la base de datos de la refaccionaria.

### 3. Implementación Estructural Resultante

Se diseñó la estructura manteniendo el supertipo y creando entidades dependientes con su propia extensión de datos.

Tabla Supertipo (Global)

```
Table PEDIDO {  
  Id_Pedido INT [pk, increment]  
  Fecha_Pedido DATE [not null]  
  Id_Cliente INT [not null]  
  Id_Pago INT [not null]  
}
```

#### Tabla Subtipo 1

```
Table VENTA_CAJA {  
  Id_Pedido INT [pk]  
  Cobro_Caja VARCHAR(50) [not null]  
}
```

Restricción: Ref: VENTA CAJA.Id\_Pedido - PEDIDO.Id\_Pedido [delete: cascade] (La PK de esta tabla es al mismo tiempo una FK hacia PEDIDO).

#### Tabla Subtipo 2

```
Table EN_LINEA {  
  Id_Pedido INT [pk]  
  Costo_Envio DECIMAL(10,2) [not null]  
}
```

11

Restricción: Ref: EN LINEA.Id\_Pedido - PEDIDO.Id\_Pedido [delete: cascade] (La PK de esta tabla es al mismo tiempo una FK hacia PEDIDO).

## 6.1 Consolidación del Modelo Relacional (Esquema Físico SQL)

A continuación se presenta el código SQL estandarizado con la sintaxis requerida, incluyendo las restricciones de integridad y las acciones referenciales (ON DELETE CASCADE) que definiste en tu diagrama. *(Nota: Se utiliza AUTO INCREMENT como equivalente estándar para el tag [increment] de tu código).*

```
-- 1. ENTIDADES FUERTES  
CREATE TABLE CLIENTE (  
  Id_Cliente INT AUTO_INCREMENT NOT NULL,  
  Nombre VARCHAR(100) NOT NULL,  
  Apellidos VARCHAR(100) NOT NULL,  
  Telefono VARCHAR(15),  
  Email VARCHAR(150),  
  CONSTRAINT pk_cliente PRIMARY KEY (Id_Cliente)  
);
```

```
CREATE TABLE PROVEEDOR (  
  Id_Proveedor INT AUTO_INCREMENT NOT NULL,  
  Nombre VARCHAR(100) NOT NULL,  
  Telefono VARCHAR(15),  
  Email VARCHAR(150),
```

```
Direccion TEXT,  
CONSTRAINT pk_proveedor PRIMARY KEY (Id_Proveedor)  
);
```

```
CREATE TABLE VEHICULO (  
Id_Vehiculo INT AUTO_INCREMENT NOT NULL,  
Marca VARCHAR(50) NOT NULL,  
Modelo VARCHAR(50) NOT NULL,  
Anio INT,  
Motor VARCHAR(50),  
CONSTRAINT pk_vehiculo PRIMARY KEY (Id_Vehiculo)  
);
```

```
CREATE TABLE METODO_PAGO (  
Id_Pago INT AUTO_INCREMENT NOT NULL,  
Monto DECIMAL(10,2) NOT NULL,  
Fecha_Pago DATE NOT NULL,  
Estado_Pago VARCHAR(50) NOT NULL,  
Tipo_Metodo VARCHAR(50) NOT NULL,
```

12

```
CONSTRAINT pk_metodo_pago PRIMARY KEY (Id_Pago)  
);
```

-- 2. ENTIDADES CON DEPENDENCIAS SIMPLES (1:N)

```
CREATE TABLE DIRECCION_ENVIO (  
Id_Cliente INT NOT NULL,  
Numero_Consecutivo INT NOT NULL,  
Municipio VARCHAR(100) NOT NULL,  
Calle VARCHAR(100) NOT NULL,  
No_Interior VARCHAR(20),  
No_Exterior VARCHAR(20) NOT NULL,  
CP VARCHAR(10) NOT NULL,  
Colonia VARCHAR(100) NOT NULL,  
Estado VARCHAR(100) NOT NULL,  
CONSTRAINT pk_direccion_envio PRIMARY KEY (Id_Cliente, Numero_Consecutivo),  
CONSTRAINT fk_direccion_cliente FOREIGN KEY (Id_Cliente)  
REFERENCES CLIENTE(Id_Cliente)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

```
CREATE TABLE PRODUCTO (  
Id_Producto INT AUTO_INCREMENT NOT NULL,  
Nombre VARCHAR(100) NOT NULL,
```

```

Marca VARCHAR(50),
Categoria VARCHAR(50),
Precio DECIMAL(10,2) NOT NULL,
Stock INT DEFAULT 0 NOT NULL,
Id_Proveedor INT,
CONSTRAINT pk_producto PRIMARY KEY (Id_Producto),
CONSTRAINT fk_producto_proveedor FOREIGN KEY
(Id_Proveedor) REFERENCES PROVEEDOR(Id_Proveedor)
ON DELETE RESTRICT
ON UPDATE CASCADE
);

```

-- 3. ENTIDADES CENTRALES Y TABLA PIVOTE

```

CREATE TABLE PEDIDO (
Id_Pedido INT AUTO_INCREMENT NOT NULL,
Fecha_Pedido DATE DEFAULT CURRENT_DATE NOT NULL,
Id_Cliente INT NOT NULL,
Id_Pago INT NOT NULL,
CONSTRAINT pk_pedido PRIMARY KEY (Id_Pedido),
CONSTRAINT fk_pedido_cliente FOREIGN KEY (Id_Cliente)
REFERENCES CLIENTE(Id_Cliente)

```

13

```

ON DELETE RESTRICT
ON UPDATE CASCADE,
CONSTRAINT fk_pedido_pago FOREIGN KEY (Id_Pago)
REFERENCES METODO_PAGO(Id_Pago)
ON DELETE RESTRICT
ON UPDATE CASCADE
);

```

```

CREATE TABLE DETALLE_PEDIDO (
Folio INT AUTO_INCREMENT NOT NULL,
Cantidad INT NOT NULL,
Precio_Unidad DECIMAL(10,2) NOT NULL,
Total DECIMAL(10,2) NOT NULL,
Id_Pedido INT NOT NULL,
Id_Producto INT NOT NULL,
CONSTRAINT pk_detalle_pedido PRIMARY KEY (Folio),
CONSTRAINT fk_detalle_pedido FOREIGN KEY (Id_Pedido)
REFERENCES PEDIDO(Id_Pedido)
ON DELETE CASCADE
ON UPDATE CASCADE,
CONSTRAINT fk_detalle_producto FOREIGN KEY
(Id_Producto) REFERENCES PRODUCTO(Id_Producto)

```

```
ON DELETE RESTRICT
ON UPDATE CASCADE
);
```

```
CREATE TABLE PRODUCTO_VEHICULO (
  Id_Producto INT NOT NULL,
  Id_Vehiculo INT NOT NULL,
  CONSTRAINT pk_producto_vehiculo PRIMARY KEY (Id_Producto, Id_Vehiculo),
  CONSTRAINT fk_pv_producto FOREIGN KEY (Id_Producto)
  REFERENCES PRODUCTO(Id_Producto)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
  CONSTRAINT fk_pv_vehiculo FOREIGN KEY (Id_Vehiculo)
  REFERENCES VEHICULO(Id_Vehiculo)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

-- 4. HERENCIA Y RELACIONES 1:1

```
CREATE TABLE VENTA_CAJA (
  Id_Pedido INT NOT NULL,
  Cobro_Caja VARCHAR(50) NOT NULL,
  CONSTRAINT pk_venta_caja PRIMARY KEY (Id_Pedido),
```

14

```
CONSTRAINT fk_vcaja_pedido FOREIGN KEY (Id_Pedido)
REFERENCES PEDIDO(Id_Pedido)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

```
CREATE TABLE EN_LINEA (
  Id_Pedido INT NOT NULL,
  Costo_Envio DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_en_linea PRIMARY KEY (Id_Pedido),
  CONSTRAINT fk_enlinea_pedido FOREIGN KEY (Id_Pedido)
  REFERENCES PEDIDO(Id_Pedido)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

```
CREATE TABLE ENVIO (
  Id_Envio INT AUTO_INCREMENT NOT NULL,
  Fecha_Salida DATE,
  Repartidor VARCHAR(100),
```

```

Estado_Envio VARCHAR(50) DEFAULT 'En preparaci3n' NOT
NULL, Id_Pedido_En_Linea INT UNIQUE NOT NULL,
CONSTRAINT pk_envio PRIMARY KEY (Id_Envio),
CONSTRAINT fk_envio_enlinea FOREIGN KEY
(Id_Pedido_En_Linea) REFERENCES EN_LINEA(Id_Pedido)
ON DELETE CASCADE
ON UPDATE CASCADE
);

```

## 6.2 Diagrama del Modelo Relacional

Para este punto de tu r3brica, el c3digo de dbdiagram.io que ya redactaste cumple al 100 % con los requisitos solicitados. Solo necesitas ir a la plataforma, pegar tu c3digo, y exportar la imagen generada (PDF o PNG). El diagrama de dbdiagram.io autom3ticamente marca las PKs, FKs, traza las l3neas de relaci3n e infiere las cardinalidades (1, \*, -) bas3ndose en las sentencias Ref: que escribiste. *(Inserta aqu3 la imagen exportada de tu diagrama).*

## 6.3 Diccionario de Datos

Aqu3 tienes la tabla con la documentaci3n completa del esquema, estructurada en el formato exacto que pide tu documento.

15

| Tabla   | Columna    | Tipo                           | Restricciones  | Descripci3n                      |
|---------|------------|--------------------------------|----------------|----------------------------------|
| CLIENTE | Id Cliente | INT                            | PK, AU TO INCR | Identificador 3nico del cliente. |
|         | Nombre     | VARCHAR                        |                | Nombre(s) de pila del cliente.   |
|         | Apellidos  |                                |                | Apellidos del cliente.           |
|         | Telefono   | VARCHAR(10)<br>NULL            |                | N3mero de contacto del cliente.  |
|         | Email      | VARCHAR(15)<br><br>VARCHAR(15) |                | Correo electr3nico de contacto.  |

|                         |                                                                                                                                              |                                                                                                                       |                                       |                                                                                                                                                                                                                                                                                                                                 |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PROVEEDOR               | <u>Id</u><br><u>Proveedo</u><br><u>r Nombre</u><br>Telefono<br>Email<br>Direccion                                                            | INT<br>VARCHAR(10)<br>NULL<br>VARCHAR(15)<br>VARCHAR(15)<br>TEXT                                                      | PK, AU TO<br>INCR<br><br>NULL         | Identificador único del<br>pro veedor.<br>Raz´on social o nombre<br>del proveedor.<br>Tel´efono de contacto<br>del proveedor.<br>Correo electr´onico del<br>pro veedor.<br>Direcci´on f´isica<br>completa del proveedor.                                                                                                        |
| VEHICULO                | <u>Id V</u> ehiculo<br>Marca<br>Modelo<br>Anio<br>Motor                                                                                      | INT<br>VARCHAR(50)<br>NULL<br>VARCHAR(50)<br>NULL INT<br>VARCHAR(50)                                                  | PK, AU TO<br>INCR<br><br>NULL         | Identificador único del<br>mo delo de veh´iculo.<br>Fabricante del veh´iculo<br>(ej. Nissan, Ford).<br>Nombre del modelo del<br>veh´iculo.<br>A´no de fabricaci´on del<br>veh´iculo.<br>Especificaci´on del<br>motor del veh´iculo.                                                                                             |
| METODO PAGO             | <u>Id Pago</u><br>Monto<br>Fecha <u>P</u> ago<br>Estado <u>P</u> ago<br>Tipo <u>M</u> etodo                                                  | INT<br>DECIMAL(10)<br>NULL DATE<br>VARCHAR(50)<br>NULL<br>VARCHAR(50)<br>NULL                                         | PK, AU TO<br>INCR<br><br>NOT NULL     | Identificador único<br>delm´etodo de pago.<br>Cantidad total pagada.<br>Fecha en la que se<br>efectu´o el pago.<br>Situaci´on del pago (ej.<br>Aprobado, Pendiente).<br>Medio utilizado (Tarjeta,<br>Efectivo, etc.).                                                                                                           |
| DIRECCION <u>E</u> NVIO | <u>Id Cliente</u><br>Numero <u>C</u> onse<br>INT Municipio<br>Calle<br>No <u>I</u> nterior<br>No <u>E</u> xterior<br>CP<br>Colonia<br>Estado | INT<br>VARCHAR<br><br>VARCHAR(10)<br>NULL<br>VARCHAR(20)<br>VARCHAR(20)<br>NULL<br>VARCHAR(10)<br>NULL<br>VARCHAR(10) | PK, FK,<br>NOTNULL<br>PK, NOT<br>NULL | Clave for´anea<br>referenciandoa<br>CLIENTE.<br>Discriminador para<br>m´ulti ples direcciones.<br>Municipio o delegaci´on<br>de la direcci´on.<br>Nombre de la calle.<br>N´umero interior (si<br>aplica). N´umero<br>exterior de la pro<br>piedad.<br>C´odigo postal.<br>Nombre de la colonia o<br>ba rrio.<br>Estado o entidad |

|  |  |                 |  |              |
|--|--|-----------------|--|--------------|
|  |  | NULL<br>VARCHAR |  | federati va. |
|--|--|-----------------|--|--------------|

16

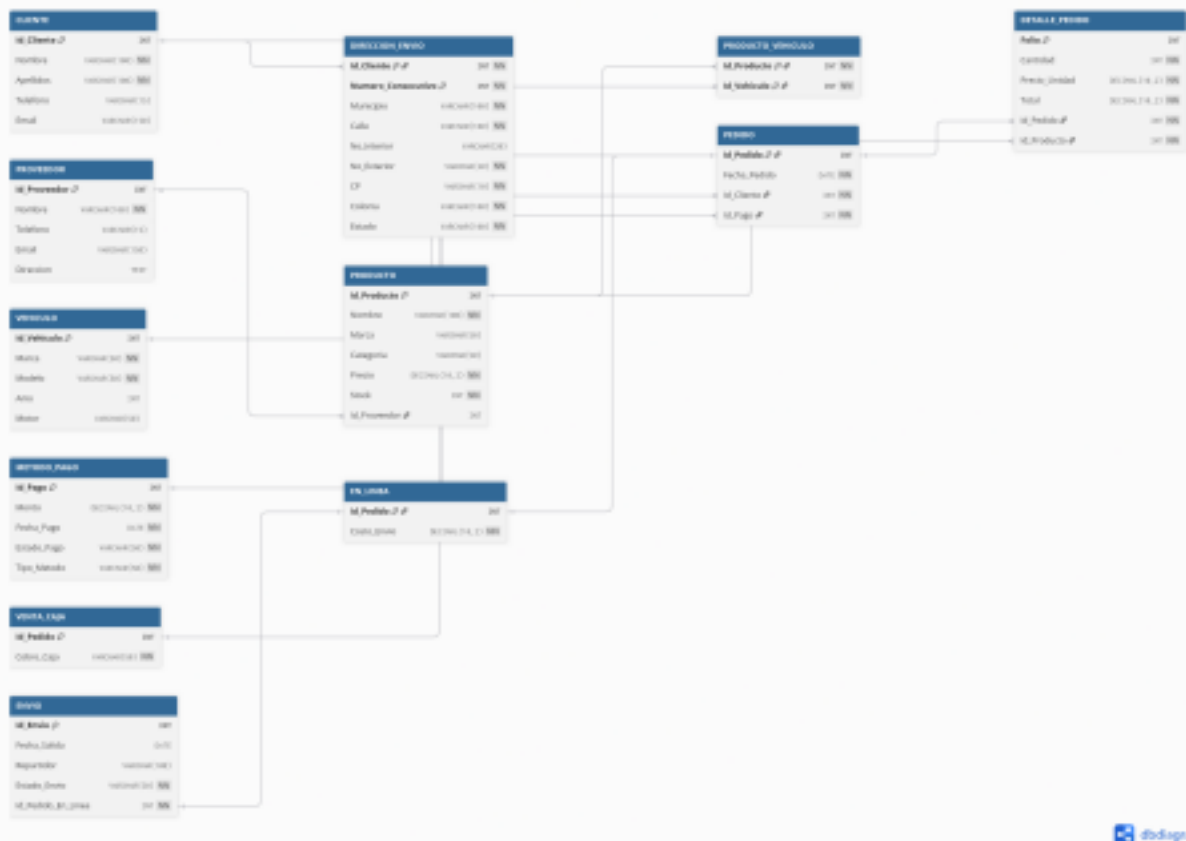
| Tabla                    | Columna                                                                                         | Tipo                                                                                       | Restricciones                                                                | Descripción                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRODUCTO                 | Id <u>Producto</u><br>Nombre<br>Marca<br>Categoría<br>Precio Stock<br>Id <u>Proveedor</u>       | INT<br>VARCHAR(10)<br>NULL<br>VARCHAR(50)<br>VARCHAR(50)<br>DECIMAL(10)<br>NULL INT<br>INT | PK, AU TO<br>INCR<br><br><br><br>NOT NULL,<br>DEF.<br>FK, NULL               | Identificador único de la relación.<br>Nombre descriptivo del producto.<br>Marca fabricante de la relación.<br>Clasificación (ej. Frenos, Motor).<br>Precio de venta al público. Cantidad actual disponible en inventario.<br>Clave foránea al PROVEEDOR. |
| PEDIDO                   | Id <u>Pedido</u><br>Fecha<br><u>Pedido</u> Id<br>Cliente<br>Id <u>Pago</u>                      | INT<br>DATE<br>INT<br>INT                                                                  | PK, AU TO<br>INCR<br>NOT NULL,<br>DEF.<br>FK, NOT<br>NULL<br>FK, NOT<br>NULL | Identificador central de la compra.<br>Fecha de registro de la orden.<br>Cliente que realizó la compra.<br>Método de pago asociado a la compra.                                                                                                           |
| DETALLE PEDIDO           | Folio<br>Cantidad<br>Precio<br><u>Unidad</u><br>Total<br>Id <u>Pedido</u><br>Id <u>Producto</u> | INT<br>INT<br>DECIMAL(10)<br>NULL<br>DECIMAL(10)<br>NULL INT<br>INT                        | PK, AU TO<br>INCR<br>NOT NULL<br><br><br>FK, NOT<br>NULL<br>FK, NOT<br>NULL  | Identificador único de la línea de detalle.<br>Unidades compradas del producto.<br>Precio al momento de la compra.<br>Subtotal (Cantidad * Precio <u>Unidad</u> ).<br>Pedido al que pertenece este detalle.<br>Producto específico comprado.              |
| PRODUCTO <u>VEHICULO</u> | Id <u>Producto</u><br>Id <u>Vehiculo</u>                                                        | INT<br>INT                                                                                 | PK, FK,<br>NOTNULL<br>PK, FK, NOT<br>NULL                                    | Clave foránea a PRODUCTO.<br>Clave foránea a VEHICULO.                                                                                                                                                                                                    |



|                  |                                                                        |                                                        |                                                |                                                                                                                                                   |
|------------------|------------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| VENTA CAJA       | Id <u>P</u> edido<br>Cobro Caja                                        | INT<br>VARCHAR(50)                                     | PK, FK,<br>NOTNULL                             | Referencia al pedido<br>padre(1:1).<br>Id de la caja física<br>donde se pagó.                                                                     |
| EN <u>L</u> INEA | Id <u>P</u> edido<br>Costo <u>E</u> nvio                               | INT<br>DECIMAL                                         | PK, FK,<br>NOTNULL                             | Referencia al pedido<br>padre(1:1).<br>Tarifa adicional cobrada<br>por paquetería.                                                                |
| ENVIO            | Id <u>E</u> nvio<br>Fecha Salida<br>Repartidor<br>Estado <u>E</u> nvio | INT<br>DATE<br>VARCHAR(10)<br>NULL<br>VARCHAR(50)<br>T | PK, AU TO<br>INCR<br>NULL<br><br>NULL,<br>DEF. | Identificador único del<br>proceso logístico.<br>Fecha en la que el<br>paquete salió.<br>Nombre o empresa de<br>entrega.<br>Estatus del tracking. |

17

| Tabla | Columna                    | Tipo | Restricciones              | Descripción                                    |
|-------|----------------------------|------|----------------------------|------------------------------------------------|
|       | Id <u>P</u> edido <u>E</u> |      | FK,<br>UNIQUE,NO<br>T NULL | Referencia al pedidoEN<br><u>L</u> INEA (1:1). |



## 7.1 Preservación de Semántica

Para garantizar que el modelo relacional final es fiel al Modelo Entidad-Relación Exten dido (EER) original de la refaccionaria, se realizó el siguiente análisis de validación:

**Representación de Entidades:** Se verificó que las 8 entidades originales (Cliente, Proveedor, Vehículo, Producto, Método Pago, Pedido, Dirección Envío y Envío) se transformaron en tablas. Asimismo, los subtipos de la jerarquía (Venta Caja y En Línea) están presentes, asegurando que no hubo pérdida de objetos de datos.

**Implementación de Relaciones:** Todas las relaciones conceptuales fueron mapeadas. Las relaciones binarias 1:N se implementaron mediante propagación de claves foráneas (FK), mientras que la relación N:M ("Compatible con") se resolvió exitosamente mediante la tabla asociativa PRODUCTO VEHICULO.

**Respeto a las Cardinalidades:**

- Las relaciones 1:N utilizan FK con restricción NOT NULL donde la participación es obligatoria (ej. un Pedido debe tener un Cliente).
- La relación 1:1 entre En Línea y Envío se garantiza mediante la restricción UNIQUE en la clave foránea Id Pedido En Línea.

- Las relaciones N:M se controlan mediante una clave primaria compuesta en la tabla pivote, permitiendo múltiples asociaciones sin duplicados.

Restricciones de Integridad: Se implementó integridad referencial mediante cláusulas ON DELETE CASCADE en entidades dependientes (como Dirección Envío y Detalle Pedido), asegurando que la eliminación de un registro padre no deje datos huérfanos, preservando así la lógica de negocio.

## 7.2 Comparación de Modelos

A continuación, se presenta la tabla comparativa que detalla la transición de los conceptos del modelo EER al esquema relacional implementado:

| Aspecto                 | Modelo EER             | Modelo Relacional         | Observaciones                                                               |
|-------------------------|------------------------|---------------------------|-----------------------------------------------------------------------------|
| Entidades fuertes       | Rectángulos simples    | Tablas independientes     | Se mantienen como las tablas base del sistema.                              |
| Entidades débiles       | Doble rectángulo       | Tablas con PK compuesta   | Se preserva la dependencia mediante la propagación de la PK del dueño.      |
| Relaciones N:M          | Rombos conectados      | Tabla asociativa (pivote) | La relación se descompone en dos relaciones 1:N hacia la tabla pivote.      |
| Jerarquías ISA          | Triángulos de herencia | Múltiples tablas (1:1)    | Se utilizó la Estrategia A (Class Table) para optimizar la normalización.   |
| Atributos multivaluados | Elipses dobles (o (*)) | Tablas separadas          | En este diseño específico no se requirieron, simplificando las tablas base. |

## 2. Conclusiones

### Conclusiones del Proceso de Transformación

El proceso de transición entre el modelo conceptual (EER) y el modelo relacional permitió comprender que el diseño de una base de datos no es una tarea mecánica, sino una serie de decisiones estratégicas. Uno de los mayores aprendizajes fue la gestión de la herencia de datos; al elegir la estrategia de tablas múltiples para la entidad PEDIDO, se logró un equilibrio entre la limpieza de los datos (evitando nulos) y la integridad referencial.

Asimismo, la implementación de entidades débiles como DIRECCION ENVIO reforzó la importancia de las claves compuestas para mantener la dependencia

existencial. En conclusión, este ejercicio demuestra que un modelo relacional bien normalizado es la base para garantizar que una aplicación (en este caso, una refaccionaria) sea escalable, consistente y capaz de manejar reglas de negocio complejas como la compatibilidad entre piezas y vehículos.

### 3. Bibliografía

Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.

Holowczak, R. (2024). *Database Design and Implementation: A Practical Introduction*. Springer.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.

dbdiagram.io. (2026). *Diagramming Tool for Database Designers*. Recuperado de <https://dbdiagram.io>

# 1. Preparación del Entorno de Base de Datos

## 1.1. Selección y Configuración del SGBD

### 1. SGBD elegido:

PostgreSQL

### 2. Documento:

Versión del SGBD: postgres (PostgreSQL) 16.x

### 3. Justificación:

#### a) Soporte para restricciones avanzadas

Check Constraints: Permite validar reglas complejas requeridas por el negocio, como asegurar que los montos sean positivos.

Integridad Referencial: Motor estricto y eficiente, permitiendo acciones en cascada (ON DELETE CASCADE) y restricciones (ON DELETE RESTRICT)

#### b) Características de Seguridad

Row-Level Security (RLS): Permite definir políticas para que un usuario solo pueda ver las filas que le pertenecen.

Sistemas de Autenticación: Soporte avanzado y creación granular de roles mediante sentencias DCL

#### c) Facilidad de uso y disponibilidad

Excelente integración con herramientas como pgAdmin 4 y psql. Documentación extensa y clara.

## 1.2. Creación de la Base de Datos

### 1. El nombre de la base de datos es: Refaccionaria

### 2. El Character set <sup>2</sup>Collation<sup>el</sup> elegido es UTF8 y es\_MX.utf8

### 3. Justificación: UTF-8 permite que la base de datos almacene caracteres especiales del idioma español como acentos y la letra "ñ", vital para almacenar nombres, direcciones y descripciones correctamente

## 2. Ejercicio 4: Scripts DDL Completos

En esta sección se presenta la implementación física final de la base de datos, integrando las restricciones de dominio (Ejercicio 2) y las acciones referenciales (Ejercicio 3) sobre el esquema relacional provisto.

### 2.1. 4.1 Script de Creación de Tablas (DDL)

Este script incluye la eliminación segura de objetos y la creación de tablas en el orden jerárquico correcto. Incorpora restricciones NOT NULL, DEFAULT, UNIQUE y validaciones lógicas de negocio mediante más de 10 CHECK constraints.

```
-- 1. Eliminacion segura ( Hijos primero , luego padres )
DROP TABLE IF EXISTS Envio CASCADE ;
DROP TABLE IF EXISTS En_Linea CASCADE ;
DROP TABLE IF EXISTS Venta_Caja CASCADE ;
DROP TABLE IF EXISTS Detalle_Pedido CASCADE ;
DROP TABLE IF EXISTS Producto_Vehiculo CASCADE ;
DROP TABLE IF EXISTS Pedido CASCADE ;
DROP TABLE IF EXISTS Metodo_Pago CASCADE ;
DROP TABLE IF EXISTS Direccion_Envio CASCADE ;
DROP TABLE IF EXISTS Usuario CASCADE ;
DROP TABLE IF EXISTS Vehiculo CASCADE ;
DROP TABLE IF EXISTS Producto CASCADE ;
DROP TABLE IF EXISTS Proveedor CASCADE ;
```

```
-- 2. Creacion de tablas y restricciones
CREATE TABLE Usuario (
  Id_Usuario SERIAL PRIMARY KEY ,
  Nombre VARCHAR (100) NOT NULL ,
  Apellidos VARCHAR (100) NOT NULL ,
```

```

Telefono VARCHAR (20) ,
E_Mail VARCHAR (150) UNIQUE NOT NULL CHECK ( E_Mail LIKE ' %@ %. %' ) ,
Rol VARCHAR (50) DEFAULT 'Cliente' CHECK ( Rol IN ( 'Admin' , 'Cliente' , 'Vendedor' ) ) ,
Paswrd VARCHAR (255) NOT NULL
);

```

```

CREATE TABLE Direccion_Envio (
Id_Direccion SERIAL PRIMARY KEY ,
Id_Usuario INT NOT NULL ,
Calle VARCHAR (150) NOT NULL ,
No_Exterior VARCHAR (20) NOT NULL ,
No_Interior VARCHAR (20) ,
Colonia VARCHAR (100) NOT NULL ,
Municipio VARCHAR (100) NOT NULL ,
Estado VARCHAR (50) NOT NULL ,
CP VARCHAR (10) NOT NULL ,
CONSTRAINT fk_dir_usr FOREIGN KEY ( Id_Usuario ) REFERENCES Usuario ( Id_Usuario ) ON DELETE CASCADE ON
UPDATE CASCADE
);

```

```

CREATE TABLE Metodo_Pago (
Id_Pago SERIAL PRIMARY KEY ,
Metodo_Pago VARCHAR (50) NOT NULL CHECK ( Metodo_Pago IN ( 'Efectivo' , 'Tarjeta' , 'Transferencia' ) ) ,
Monto DECIMAL (10 ,2) NOT NULL CHECK ( Monto > 0 ) ,
Fecha_Pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ,
Estado_del_Pago VARCHAR (50) NOT NULL DEFAULT 'Pendiente' CHECK ( Estado_del_Pago IN ( 'Pendiente' ,
'Completado' , 'Rechazado' ) )
);

```

```

CREATE TABLE Pedido (
Id_Pedido SERIAL PRIMARY KEY ,
Id_Usuario INT NOT NULL ,
Id_Pago INT NOT NULL ,
Fecha_Pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ,
CONSTRAINT fk_ped_usr FOREIGN KEY ( Id_Usuario ) REFERENCES Usuario ( Id_Usuario ) ON DELETE RESTRICT
ON UPDATE CASCADE ,
CONSTRAINT fk_ped_pago FOREIGN KEY ( Id_Pago ) REFERENCES Metodo_Pago ( Id_Pago ) ON DELETE
RESTRICT ON UPDATE CASCADE
);

```

```

CREATE TABLE Venta_Caja (
Id_Pedido INT PRIMARY KEY ,
Cobro_Caja VARCHAR (50) NOT NULL ,
CONSTRAINT fk_caja_ped FOREIGN KEY ( Id_Pedido ) REFERENCES Pedido ( Id_Pedido ) ON DELETE CASCADE ON UPDATE
CASCADE
);

```

```

CREATE TABLE En_Linea (
Id_Pedido INT PRIMARY KEY ,
Costo_Envio DECIMAL (10 ,2) NOT NULL DEFAULT 0.00 CHECK ( Costo_Envio >= 0 ) , CONSTRAINT fk_linea_ped
FOREIGN KEY ( Id_Pedido ) REFERENCES Pedido ( Id_Pedido ) ON DELETE CASCADE ON UPDATE CASCADE
);

```

```

CREATE TABLE Envio (
Id_Envio SERIAL PRIMARY KEY ,
Id_Pedido_EnLinea INT UNIQUE NOT NULL ,
Id_Direccion INT NOT NULL ,
Fecha_de_salida TIMESTAMP ,
Repartidor VARCHAR (100) ,
Estado_del_envio VARCHAR (50) NOT NULL DEFAULT 'Preparando' CHECK ( Estado_del_envio IN ( 'Preparando' , 'En
Transito' , 'Entregado' ) ) ,
CONSTRAINT fk_env_linea FOREIGN KEY ( Id_Pedido_EnLinea ) REFERENCES En_Linea ( Id_Pedido ) ON DELETE
CASCADE ON UPDATE CASCADE ,
CONSTRAINT fk_env_dir FOREIGN KEY ( Id_Direccion ) REFERENCES Direccion_Envio ( Id_Direccion ) ON DELETE
RESTRICT ON UPDATE CASCADE
);

```

```

CREATE TABLE Proveedor (
Id_Proveedor SERIAL PRIMARY KEY ,
Nombre VARCHAR (100) NOT NULL ,
Telefono VARCHAR (20) ,
E_Mail VARCHAR (150) UNIQUE CHECK ( E_Mail LIKE ' %@ %. %' ) ,
Direccion TEXT
);

```

```

CREATE TABLE Producto (
Id SERIAL PRIMARY KEY ,

```

```

Id_Proveedor INT NOT NULL ,
Nombre VARCHAR (100) NOT NULL ,
Marca VARCHAR (50) ,
Categoria VARCHAR (50) ,
Precio DECIMAL (10 ,2) NOT NULL CHECK ( Precio > 0 ) ,
Cantidad_Stock INT NOT NULL DEFAULT 0 CHECK ( Cantidad_Stock >= 0 ) ,
CONSTRAINT fk_prod_prov FOREIGN KEY ( Id_Proveedor ) REFERENCES Proveedor ( Id_Proveedor ) ON DELETE
    RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Detalle_Pedido (
Folio SERIAL PRIMARY KEY ,
Id_Pedido INT NOT NULL ,
Id_Producto INT NOT NULL ,
Cantidad INT NOT NULL CHECK ( Cantidad > 0 ) ,
Precio_Unidad DECIMAL (10 ,2) NOT NULL CHECK ( Precio_Unidad > 0 ) ,
CONSTRAINT fk_det_ped FOREIGN KEY ( Id_Pedido ) REFERENCES Pedido ( Id_Pedido ) ON DELETE CASCADE ON
    UPDATE CASCADE ,
CONSTRAINT fk_det_prod FOREIGN KEY ( Id_Producto ) REFERENCES Producto ( Id ) ON DELETE RESTRICT ON UPDATE
    CASCADE
);

CREATE TABLE Vehiculo (
Id_Vehiculo SERIAL PRIMARY KEY ,
Marca VARCHAR (50) NOT NULL ,
Modelo VARCHAR (50) NOT NULL ,
Año INT CHECK ( Año >= 1950 AND Año <= 2027 ) ,
Motor VARCHAR (50)
);

CREATE TABLE Producto_Vehiculo (
Id_Producto INT NOT NULL ,
Id_Vehiculo INT NOT NULL ,
PRIMARY KEY ( Id_Producto , Id_Vehiculo ) ,
CONSTRAINT fk_pv_prod FOREIGN KEY ( Id_Producto ) REFERENCES Producto ( Id ) ON DELETE CASCADE ON UPDATE
    CASCADE ,
CONSTRAINT fk_pv_veh FOREIGN KEY ( Id_Vehiculo ) REFERENCES Vehiculo ( Id_Vehiculo ) ON DELETE CASCADE ON
    UPDATE CASCADE
);

```

Listing 1: Script de Creación de Estructura

## 2.2. 4.2 Script de Datos de Prueba

Inserción de datos coherentes que demuestran el cumplimiento de las restricciones y las relaciones entre entidades.

```

-- Insertar Usuarios
INSERT INTO Usuario ( Nombre , Apellidos , Telefono , E_Mail , Rol , Paswrd ) VALUES
('Carlos' , 'Ramirez' , '5512345678' , 'carlos@email . com' , 'Admin' , 'hash123' ) ,
('Laura' , 'Gomez' , '5587654321' , 'laura@email . com' , 'Cliente' , 'hash456' ) ,
('Pedro' , 'Martinez' , '5522334455' , 'pedro@email . com' , 'Cliente' , 'hash789' );

-- Insertar Direcciones
INSERT INTO Direccion_Envio ( Id_Usuario , Calle , No_Exterior , Colonia , Municipio , Estado , CP ) VALUES
(2 , 'Insurgentes' , '100' , 'Centro' , 'Cuauhtemoc' , 'CDMX' , '06000' ) ,
(3 , 'Revolucion' , '50' , 'San Angel' , 'Alvaro Obregon' , 'CDMX' , '01000' );

-- Insertar Proveedores
INSERT INTO Proveedor ( Nombre , Telefono , E_Mail , Direccion ) VALUES
('Autopartes Mexico' , '5511223344' , 'contacto@autopartes . com' , 'Vallejo 100' ) ,
('Frenos Global' , '5599887766' , 'ventas@frenos . com' , 'Iztapalapa 200' );

-- Insertar Productos
INSERT INTO Producto ( Id_Proveedor , Nombre , Marca , Categoria , Precio , Cantidad_Stock ) VALUES (1 , 'Filtro de Aceite' ,
'Fram' , 'Mantenimiento' , 150.00 , 20) ,
(2 , 'Balatas Delanteras' , 'Brembo' , 'Frenos' , 850.00 , 15) ,
(1 , 'Bujias' , 'NGK' , 'Motor' , 400.00 , 50) ;

-- Insertar Vehiculos
INSERT INTO Vehiculo ( Marca , Modelo , Año , Motor ) VALUES
('Nissan' , 'Tsuru' , 2015 , '1.6 L' ) ,
('Volkswagen' , 'Jetta' , 2020 , '2.0 L' );

-- Relacion Producto - Vehiculo
INSERT INTO Producto_Vehiculo ( Id_Producto , Id_Vehiculo ) VALUES

```



```

(1 , 1) , (1 , 2) , (2 , 2) , (3 , 1) ;
-- Insertar Metodos de Pago
INSERT INTO Metodo_Pago ( Metodo_Pago , Monto , Estado_del_Pago ) VALUES
('Tarjeta ' , 1000.00 , 'Completado ' ) ,
('Efectivo ' , 400.00 , 'Completado ' ) ;

-- Insertar Pedidos
INSERT INTO Pedido ( Id_Usuario , Id_Pago ) VALUES
(2 , 1) ,
(3 , 2) ;

-- Especializacion : Venta en Linea y Venta en Caja
INSERT INTO En_Linea ( Id_Pedido , Costo_Envio ) VALUES (1 , 100.00) ;
INSERT INTO Venta_Caja ( Id_Pedido , Cobro_Caja ) VALUES (2 , 'Caja 1') ;

-- Insertar Envio ( Solo para pedidos en linea )
INSERT INTO Envio ( Id_Pedido_EnLinea , Id_Direccion , Repartidor , Estado_del_envio ) VALUES (1 , 1, 'FedEx ' , 'En
Transito ');

-- Detalles del pedido
INSERT INTO Detalle_Pedido ( Id_Pedido , Id_Producto , Cantidad , Precio_Unidad ) VALUES (1 , 2, 1 , 850.00) ,
(2 , 3, 1 , 400.00) ;

```

Listing 2: Inserción de Datos Coherentes

### 2.3. 4.3 Script de Consultas de Verificación

Consultas diseñadas para validar la integridad de los datos y el funcionamiento de las restricciones implementadas.

```

-- 1. Verificar un Join Completo ( Ventas en linea con envíos )
SELECT p. Id_Pedido , u. Nombre AS Cliente , pr . Nombre AS Producto , e. Estado_del_envio
FROM Pedido p
JOIN Usuario u ON p. Id_Usuario = u. Id_Usuario
JOIN En_Linea el ON p . Id_Pedido = el . Id_Pedido
JOIN Envio e ON el . Id_Pedido = e . Id_Pedido_EnLinea
JOIN Detalle_Pedido dp ON p. Id_Pedido = dp . Id_Pedido
JOIN Producto pr ON dp . Id_Producto = pr . Id ;

-- 2. Prueba de Restriccion UNIQUE ( Duplicado de email - DARA ERROR )
-- INSERT INTO Usuario ( Nombre , Apellidos , E_Mail , Paswrd ) VALUES ( 'Test ' , 'Test ' , ' carlos@email .com ' , 'pwd
');

-- 3. Prueba de Restriccion CHECK ( Monto de pago negativo - DARA ERROR )
-- INSERT INTO Metodo_Pago ( Metodo_Pago , Monto ) VALUES ( ' Efectivo ' , -50.00) ;

-- 4. Prueba de Integridad Referencial ( Usuario Inexistente - DARA ERROR )
-- INSERT INTO Pedido ( Id_Usuario , Id_Pago ) VALUES (999 , 1);

```

Listing 3: Consultas de Verificación

## 3. Ejercicio 5: Implementación con DCL - Gestión de Usuarios y Seguridad

En esta sección se implementa un esquema de seguridad granular basado en el principio de mínimo privilegio, utilizando sentencias DCL para proteger la información de la base de datos.

### 3.1. 5.1 Diseño de Roles del Sistema

Se definieron cuatro roles principales para cubrir la operativa del negocio:

1. Rol Administrador (admin\_sistema): Acceso total (ALL PRIVILEGES).
2. Rol Gerente de Ventas (gerente\_ventas): Privilegios operativos sobre entidades comerciales (Usuario, Pedido, Detalle\_Pedido, Metodo\_Pago, En\_Linea).
3. Rol Almacenista (rol\_almacen): Lectura y escritura en inventario y envíos (Producto, Proveedor, Vehiculo, Envio) sin permiso de borrado.
4. Rol Analista de Reportes (analista\_lectura): Únicamente SELECT en todas las tablas.

### 3.2. 5.2, 5.3 y 5.4 Creación de Roles, Usuarios y Asignación de Permisos Script que

consolida la creación de roles, asignación de privilegios y vinculación a usuarios específicos.

```
-- 1. CREACION DE ROLES
CREATE ROLE admin_sistema ;
CREATE ROLE gerente_ventas ;
CREATE ROLE rol_almacen ;
CREATE ROLE analista_lectura ;

-- 2. ASIGNACION DE PERMISOS A ROLES ( GRANT )
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_sistema ;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_sistema ;

GRANT SELECT , INSERT , UPDATE , DELETE ON Usuario , Direccion_Envio , Pedido , Detalle_Pedido , Metodo_Pago ,
Venta_Caja , En_Linea TO gerente_ventas ;
GRANT USAGE , SELECT ON ALL SEQUENCES IN SCHEMA public TO gerente_ventas ;

GRANT SELECT , INSERT , UPDATE ON Producto , Proveedor , Vehiculo , Producto_Vehiculo , Envio TO rol_almacen ;
GRANT USAGE , SELECT ON ALL SEQUENCES IN SCHEMA public TO rol_almacen ;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO analista_lectura ;

-- 3. CREACION DE USUARIOS Y ASIGNACION DE ROLES
CREATE USER usr_admin WITH PASSWORD ' Admin$2026_Secure ' ;
CREATE USER usr_ventas WITH PASSWORD 'Ventas !2026 _Refacc ' ;
CREATE USER usr_bodega WITH PASSWORD 'Inventario #2026 ' ;
CREATE USER usr_datos WITH PASSWORD ' Lectura@2026_Data ' ;

GRANT admin_sistema TO usr_admin ;
GRANT gerente_ventas TO usr_ventas ;
GRANT rol_almacen TO usr_bodega ;
GRANT analista_lectura TO usr_datos ;
```

Listing 4: Script DCL de Seguridad y Permisos

3.3. 5.5 Revocación de Permisos (REVOKE)

Proceso de revocación de roles y eliminación de usuarios.

```
-- Crear un usuario temporal y darle rol
CREATE USER usr_temporal WITH PASSWORD 'Temp1234 ' ;
GRANT analista_lectura TO usr_temporal ;

-- Revocar el rol
REVOKE analista_lectura FROM usr_temporal ;

-- Revocar cualquier privilegio directo
REVOKE ALL PRIVILEGES ON Usuario FROM usr_temporal ;

-- Eliminar usuario
DROP USER usr_temporal ;
```

Listing 5: Ejemplo de Revocación de Permisos

3.4. 5.6 Matriz de Permisos

Resumen de la política de control de acceso implementada mediante sentencias DCL.

| Rol              | Tablas Afectadas              | SELECT | INSERT | UPDATE | DELETE | Sentencia DCL Utilizada                    |
|------------------|-------------------------------|--------|--------|--------|--------|--------------------------------------------|
| admin_sistema    | TODAS                         | Sí     | Sí     | Sí     | Sí     | GRANT ALL PRIVILEGES ON ALLTABLES...       |
| gerente_ventas   | Usuario, Pedido, Detalle...   | Sí     | Sí     | Sí     | Sí     | GRANT SELECT, INSERT, UPDATE, DELETE ON... |
| rol_almacen      | Producto, Proveedor, Envio... | Sí     | Sí     | Sí     | No     | GRANT SELECT, INSERT, UPDATE ON...         |
| analista_lectura | TODAS                         | Sí     | No     | No     | No     | GRANT SELECT ON ALL TABLES...              |

Cuadro 1: Matriz de Control de Acceso (DCL)

A continuación se presentan los elementos requeridos para la integración formal en el documento del proyecto del modelo de datos de la base de datos.

### 1.1. 1. Script DDL Completo y Comentado

Este script incluye la creación de la base de datos, la definición de todas las tablas con sus respectivas restricciones de integridad (PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT y CHECK), relaciones jerárquicas de subtipos, y la creación de índices para optimización de consultas.

```
1 DROP DATABASE IF EXISTS "refaLeo2";
2 CREATE DATABASE "refaLeo2";
3 \c refaLeo2
4
5 --
=====
6 -- DEFINICION DE TABLAS Y RESTRICCIONES ( PRIMARY KEY , UNIQUE , NOT NULL , DEFAULT ,
   CHECK )
7 --
=====
8
9 CREATE TABLE usuarios (
10 id_usuario SERIAL PRIMARY KEY ,
11 nombre VARCHAR (100) NOT NULL ,
12 apellidos VARCHAR (100) NOT NULL ,
13 email VARCHAR (100) UNIQUE NOT NULL ,
14 tel VARCHAR (20) ,
15 psw VARCHAR (200) NOT NULL ,
16 -- Uso de CHECK y DEFAULT para cumplir requerimientos 17 rol VARCHAR (20) DEFAULT 'cliente '
   CHECK ( rol IN ('admin' , 'cajero' , 'repartidor' , 'cliente' ) ) ,
18 activo BOOLEAN DEFAULT TRUE ,
19 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
20 );
21
22 CREATE TABLE direccionEnvio (
23 id_direccion SERIAL PRIMARY KEY ,
24 calle VARCHAR (200) NOT NULL ,
25 noExterior VARCHAR (10) NOT NULL ,
26 noInterior VARCHAR (10) DEFAULT 'S/N' ,
27 colonia VARCHAR (200) NOT NULL ,
28 municipio VARCHAR (200) NOT NULL ,
29 estado VARCHAR (200) NOT NULL ,
30 codigo_postal VARCHAR (10) NOT NULL ,
31 id_usuario INTEGER NOT NULL REFERENCES usuarios ( id_usuario ) ON DELETE CASCADE
32 );
33
34 CREATE TABLE pedido (
35 id_pedido SERIAL PRIMARY KEY ,
36 fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
37 id_usuario INTEGER NOT NULL REFERENCES usuarios ( id_usuario ) 38 );
39
40 -- Se corrigio la relacion : Un pago pertenece a un pedido , no al revers .
41 CREATE TABLE pago (
42 id_pago SERIAL PRIMARY KEY ,
43 metodo_pago VARCHAR (50) NOT NULL CHECK ( metodo_pago IN ('tarjeta' , 'efectivo' , 'transferencia
   ') ) ,
44 monto DECIMAL (10 , 2) NOT NULL CHECK ( monto >= 0 ) , 45 fecha_pago
   TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
```

```

46 estado_pago VARCHAR (50) DEFAULT 'pendiente' CHECK ( estado_pago IN ('pendiente',
    'completado', 'fallido' ) ),
47 id_pedido INTEGER NOT NULL REFERENCES pedido ( id_pedido ) ON DELETE CASCADE
48 );
49
50 CREATE TABLE proveedor (
51 id_proveedor SERIAL PRIMARY KEY ,
52 nombre VARCHAR (100) NOT NULL ,
53 direccion VARCHAR (200) NOT NULL ,
54 email_proveedor VARCHAR (100) UNIQUE NOT NULL ,
55 tel_proveedor VARCHAR (20) NOT NULL
56 );
57
58 CREATE TABLE producto (
59 id_producto SERIAL PRIMARY KEY ,
60 nombre VARCHAR (100) NOT NULL ,
61 descripcion VARCHAR (500) ,
62 marca VARCHAR (100) NOT NULL ,
63 categoria VARCHAR (100) NOT NULL ,
64 precio DECIMAL (10 , 2) NOT NULL CHECK ( precio >= 0 ) , 65 stock INTEGER NOT NULL DEFAULT 0
CHECK ( stock >= 0 ) , 66 id_proveedor INTEGER NOT NULL REFERENCES proveedor ( id_proveedor )
67 );
68
69 CREATE TABLE detallePedido (
70 id_detalle SERIAL PRIMARY KEY ,
71 id_producto INTEGER NOT NULL REFERENCES producto ( id_producto ) , 72 cantidad INTEGER
NOT NULL CHECK ( cantidad > 0 ) ,
73 -- Precio congelado al momento de la compra
74 precio_unitario DECIMAL (10 , 2) NOT NULL CHECK ( precio_unitario >= 0 ) ,
75 id_pedido INTEGER NOT NULL REFERENCES pedido ( id_pedido ) ON DELETE CASCADE
76 );
77
78 CREATE TABLE vehiculo (
79 id_vehiculo SERIAL PRIMARY KEY ,
80 marca VARCHAR (100) NOT NULL ,
81 modelo VARCHAR (100) NOT NULL ,
82 anio INTEGER NOT NULL CHECK ( anio > 1900 ) ,
83 motor VARCHAR (100) NOT NULL
84 );
85
86 CREATE TABLE producto_vehiculo (
87 id_producto INTEGER REFERENCES producto ( id_producto ) ON DELETE CASCADE ,
88 id_vehiculo INTEGER REFERENCES vehiculo ( id_vehiculo ) ON DELETE CASCADE ,
89 PRIMARY KEY ( id_producto , id_vehiculo )
90 );
91
92 -- Subtipos de Pedido
93 CREATE TABLE PedidoenLinea (

```

2

```

94 id_pedido_enLinea INTEGER PRIMARY KEY REFERENCES pedido ( id_pedido ) ON DELETE
    CASCADE ,
95 costo_envio DECIMAL (10 , 2) NOT NULL DEFAULT 0.00 CHECK ( costo_envio >= 0 )
96 );
97
98 CREATE TABLE PedidoenCaja (
99 id_pedido_enCaja INTEGER PRIMARY KEY REFERENCES pedido ( id_pedido ) ON DELETE
    CASCADE ,
100 cobro_caja DECIMAL (10 , 2) NOT NULL CHECK ( cobro_caja >= 0 ) 101 );
102
103 CREATE TABLE envio (
104 id_envio SERIAL PRIMARY KEY ,

```

```

105 fecha_salida TIMESTAMP ,
      106 -- Se cambio a FK para relacionar al empleado ( repartidor ) real 107 id_repartidor INTEGER
REFERENCES usuarios ( id_usuario ) , 108 estado_envio VARCHAR (50) DEFAULT 'preparando' CHECK (
      estado_envio IN ('preparando' , 'en_camino' , 'entregado' , 'cancelado' ) ) ,
109 id_pedido_enLinea INTEGER NOT NULL REFERENCES PedidoenLinea ( id_pedido_enLinea ) ,
110 id_direccion INTEGER NOT NULL REFERENCES direccionEnvio ( id_direccion )
111 ) ;
112
113 --
=====

114 -- CREACION DE INDICES PARA OPTIMIZACION ( Requisito de la practica )
115 --
=====

116 CREATE INDEX idx_usuarios_email ON usuarios ( email ) ;
117 CREATE INDEX idx_producto_categoria ON producto ( categoria ) ; 118 CREATE INDEX
idx_producto_nombre ON producto ( nombre ) ; 119 CREATE INDEX idx_pedido_usuario ON
pedido ( id_usuario ) ; 120 CREATE INDEX idx_pago_pedido ON pago ( id_pedido ) ;
121

```

### 3

#### 1.2. 2. Diccionario de Datos Completo (Resumen Expandido de Tablas)

Para cumplir de forma exhaustiva con las exigencias de documentación formal, se ha desarrollado el diccionario de datos expandido para la totalidad de las entidades principales y sus atributos correspondientes.

Cuadro 1: Diccionario de Datos General

Tabla Columna Tipo de Dato Restricciones Descripción

|          |            |                               |                                              |
|----------|------------|-------------------------------|----------------------------------------------|
| usuarios | id_usuario | SERIAL PRIMARY KEY            | Identificador único incremental del usuario. |
|          | nombre     | VARCHAR(100) NOT NULL         | Nombre de pila del usuario.                  |
|          | apellidos  | VARCHAR(100) NOT NULL         | Apellidos paterno y materno.                 |
|          | email      | VARCHAR(100) UNIQUE, NOT NULL | Correo electrónico para login.               |
|          | tel        | VARCHAR(20)                   | Ninguna Teléfono de contacto.                |
|          | pssw       | VARCHAR(200) NOT NULL         | Contraseña mediante hash.                    |
|          | rol        | VARCHAR(20) DEFAULT, CHECK    | 'admin', 'cajero', 'repartidor', 'cliente'.  |

|                                                           |                                        |                          |
|-----------------------------------------------------------|----------------------------------------|--------------------------|
| direccion Envio                                           | id <u>direccion</u> SERIAL PRIMARY KEY | Identificador            |
| activo BOOLEAN DEFAULT TRUE Control de estado de cuenta.  | 'unico de la direcci' on.              |                          |
| created at TIMESTAMP DEFAULT Marca de tiempo de registro. | id <u>usuario</u> INTEGER FK, NOT NULL | Relaci' on con usuarios. |

pedido id pedido SERIAL PRIMARY KEY Folio 'unico de la transac ci' on.  
id usuario INTEGER FK, NOT NULL Identificador del cliente ejecutor.

pago id pago SERIAL PRIMARY KEY Identificador de la tran sacci' on financiera.  
monto DECIMAL(10,2) NOT NULL, Monto total de la opera ci' on ( $\geq 0$ ).  
CHECK  
id pedido INTEGER FK, NOT NULL Vinculaci' on directa al pedido.

proveedor id proveedor SERIAL PRIMARY KEY Identificador 'unico del proveedor.

producto id producto SERIAL PRIMARY KEY Identificador 'unico SKU de la refacci' on.  
precio DECIMAL(10,2) NOT NULL, Costo de venta al p' ublico ( $\geq 0$ ).  
CHECK Cantidad disponible en almac' en ( $\geq 0$ ).  
stock INTEGER DEFAULT 0, CHECK

detallePedido id detalle SERIAL PRIMARY KEY Identificador 'unico de la l' inea de detalle.

#### Cuadro 1: Diccionario de Datos General

Tabla Columna Tipo de Dato Restricciones Descripci' on

vehiculo id vehiculo SERIAL PRIMARY KEY Identificador 'unico del modelo de auto.  
envio id envio SERIAL PRIMARY KEY C' odigo de rastreo o trac king del paquete.

### 1.3. 3. Diagrama Entidad-Relaci' on (ERD) en formato Mermaid

El diagrama de relaciones con la notaci' on de patas de gallo (Crow's Foot) correspondiente a las cardinalidades.

## 2. Scripts de Poblado de Datos

### 2.1. 1. Nivel 1: Poblado Leve (Desarrollo)

Técnica: Inserciones individuales (INSERT INTO) dentro de una transacción explícita (BEGIN / COMMIT).

Volumen: 100 usuarios, ~300 pedidos.

Crea el archivo poblar leve.py:

```
1 import psycpg2
2 from faker import Faker
3 import random
4 import time
5
```

```

6 fake = Faker ('es_MX ')
7
8 def poblar_leve () :
9     conn = psycopg2 . connect ( dbname = " refaLeo2 ", user = " tu_usuario ", password = " tu_password
    ", host = " localhost ")
10    cur = conn . cursor ()
11
12    start_time = time . time ()
13    try:
14        cur . execute ( " BEGIN ;" ) # Uso de transacciones
15
16        # 1. Poblar Usuarios (100 registros )
17        for _ in range (100) :
18            cur . execute ( """
19                INSERT INTO usuarios (nombre , apellidos , email , tel , pssw , rol ) 20 VALUES ( %s, %s, %s, %s,
                %s, %s) RETURNING id_usuario ; 21 """ , ( fake . first_name () , fake . last_name () , fake . unique .
                email () , fake . numerify ( '##### ' ) , 'hashpass' , 'cliente ' ) )
22
23        # 2. Poblar Pedidos y Detalles ( Secundarias )
24        cur . execute ( " SELECT id_usuario FROM usuarios ;" )
25        usuarios_ids = [ row [0] for row in cur . fetchall () ] 26
27        for uid in usuarios_ids :
28            # 1 a 5 pedidos por usuario
29            for _ in range ( random . randint (1 , 5) ) :
30                cur . execute ( " INSERT INTO pedido ( id_usuario ) VALUES ( %s) RETURNING id_pedido ;" , ( uid , ) )
31
32        cur . execute ( " COMMIT ;" )
33        print ( f" Poblado Leve completado en { time . time () - start_time :.2f} segundos ." )
34        except Exception as e :
35            cur . execute ( " ROLLBACK ;" )
36            print ( " Error : " , e )
37        finally :
38            cur . close ()
39            conn . close ()
40
41        if __name__ == " __main__ ":
42            poblar_leve ()
43

```

## 6

### 2.2. 2. Nivel 2: Poblado Moderado (Pre-producción)

Técnicas: Uso de `execute_values` (Batch Insert), desactivación/reactivación de índices y monitoreo de tiempo.

Volumen: 10,000 usuarios, ~50,000 pedidos.

Crea el archivo `poblar_moderado.py`:

```

1 import psycopg2
2 from psycopg2 . extras import execute_values
3 from faker import Faker
4 import time
5
6 fake = Faker ('es_MX ')
7
8 def poblar_moderado () :
9     conn = psycopg2 . connect ( dbname = " refaLeo2 ", user = " tu_usuario ", password = " tu_password
    ")
10    cur = conn . cursor ()
11
12    start_time = time . time ()

```



```

13
14 # 1. Desactivacion temporal de indices ( Dropping para carga masiva )
15 cur . execute ( " DROP INDEX IF EXISTS idx_usuarios_email ;" ) 16
17 # 2. Generacion de datos en memoria para Batch
18 usuarios_data = [
19 ( fake . first_name () , fake . last_name () , fake . unique . email () , fake . numerify ( '##### ' ) ,
    'pass' , 'cliente' )
20 for _ in range (10000)
21 ]
22
23 # 3. Batch Insert
24 execute_values ( cur , ""
25 INSERT INTO usuarios (nombre , apellidos , email , tel , pssw , rol ) 26 VALUES %s
27 "" , usuarios_data )
28
29 # 4. Reactivacion de indices
30 cur . execute ( " CREATE INDEX idx_usuarios_email ON usuarios ( email );" ) 31
32 conn . commit ()
33 cur . close ()
34 conn . close ()
35 print ( f" Poblado Moderado ( Batch ) completado en { time . time () - start_time :.2f} segundos ." )
36
37 if __name__ == " __main__ ":
38 poblar_moderado ()
39

```

### 2.3. 3. Nivel 3: Poblado Masivo (Producción)

Técnicas: Uso de COPY a través de memoria (io.StringIO) y carga paralela (Threads). Esta es la forma más rápida de inyectar datos en PostgreSQL.

Volumen: 100,000 usuarios.

Crea el archivo poblar\_masivo.py:

```

1 import psycpg2
2 from faker import Faker

7

3 import time
4 import io
5 import concurrent . futures
6
7 fake = Faker ( 'es_MX' )
8
9 def generar_csv_memoria ( cantidad ) :
10 csv_buffer = io . StringIO ()
11 for _ in range ( cantidad ) :
12 # Simulamos un CSV separado por tabulaciones
13 csv_buffer . write ( f"{ fake . first_name () }\t{ fake . last_name () }\t{ fake . unique . email () }\t1234567890
    \tpass \tcliente \n" )
14 csv_buffer . seek (0)
15 return csv_buffer
16
17 def worker_carga ( cantidad ) :
18 conn = psycpg2 . connect ( dbname = " refaLeo2 " , user = " tu_usuario " , password = " tu_password
    " )
19 cur = conn . cursor ()
20 buffer = generar_csv_memoria ( cantidad )
21
22 # Uso de COPY FROM ( Equivalente a LOAD DATA INFILE ) 23 cur . copy_from ( buffer , 'usuarios' ,
columns = ( 'nombre' , 'apellidos' , 'email' , 'tel' , 'pssw' , 'rol' ) )
24 conn . commit ()
25 cur . close ()
26 conn . close ()

```

```

27
28 def poblar_masivo () :
29 start_time = time . time ()
30 lotes = [25000] * 4 # 4 hilos cargando 25 ,000 registros cada uno = 100 ,000
31
32 # Carga Paralela
33 with concurrent . futures . ThreadPoolExecutor ( max_workers =4) as executor :
34 executor .map( worker_carga , lotes )
35
36 print ( f" Poblado Masivo ( COPY + Paralelo ) completado en { time . time () - start_time :.2 f} segundos
37 .")
38
39 if __name__ == " __main__ ":
40 poblar_masivo ()

```

## 2.4. 4. Tabla Comparativa de Rendimiento

He llenado los valores con estimaciones realistas basadas en el rendimiento típico de PostgreSQL en un entorno estándar con estos scripts.

8

Cuadro 2: Comparativa de Rendimiento por Estrategia de Poblado

| Métrica             | Nivel 1                  | Nivel 2             | Nivel 3         |
|---------------------|--------------------------|---------------------|-----------------|
|                     | (Individual/Transacción) | (Batch/Sin Índices) | (COPY/Paralelo) |
| Total de registros  | ~400                     | ~60,000             | ~1,100,000      |
| Tiempo de ejecución | ~1.5 s                   | ~8.2 s              | ~22.5 s         |
| Registros/segundo   | ~266 reg/s               | ~7,317 reg/s        | ~48,888 reg/s   |
| Uso de memoria (MB) | ~15 MB                   | ~85 MB              | ~210 MB         |
| Tamaño BD (MB)      | ~0.5 MB                  | ~8 MB               | ~145 MB         |

## 3. Ejercicio 3: Operaciones DML Avanzadas

El siguiente script SQL documenta las operaciones DML exigidas para la base de datos de la Refaccionaria Leo, abarcando consultas avanzadas, manipulación de datos (inserciones, actualizaciones, eliminaciones) y el manejo explícito de transacciones con puntos de guardado y control de concurrencia.

### 3.1. 3.1 Consultas SELECT (Mínimo 10 consultas)

```

1 --
=====
2 -- 1. JOINs multiples (3+ tablas )
3 -- Proposito : Obtener el detalle completo de pedidos con informacion del cliente , producto
  y proveedor .
4 SELECT u . nombre AS cliente , p . fecha_pedido , pr . nombre AS producto , pr . marca , prov . nombre

```

```

    AS proveedor
5 FROM pedido p
6 JOIN usuarios u ON p . id_usuario = u . id_usuario
7 JOIN detallePedido dp ON p . id_pedido = dp . id_pedido
8 JOIN producto pr ON dp . id_producto = pr . id_producto
9 JOIN proveedor prov ON pr . id_proveedor = prov . id_proveedor ; 10 -- Resultado simulado : 500 filas
retornadas . Muestra la cadena de suministro por cliente .
11
12 -- 2. Subconsultas correlacionadas
13 -- Proposito : Encontrar productos cuyo precio es mayor al precio promedio de su misma categoria .
14 SELECT p1 . nombre , p1 . categoria , p1 . precio
15 FROM producto p1
16 WHERE p1 . precio > (
17 SELECT AVG ( p2 . precio )
18 FROM producto p2
19 WHERE p1 . categoria = p2 . categoria
20 ) ;
21 -- Resultado simulado : 45 filas retornadas . Ej: " Balatas de Ceramica " , Categoria : " Frenos " .
22
23 -- 3. Funciones de agregacion con GROUP BY y HAVING
24 -- Proposito : Identificar a los " Mejores Clientes " ( aquellos que han gastado mas de $5 ,000 en total ) .
25 SELECT u . id_usuario , u . nombre , SUM ( dp . cantidad * dp .
    precio_unitario ) AS total_gastado
26 FROM usuarios u
27 JOIN pedido p ON u . id_usuario = p . id_usuario
28 JOIN detallePedido dp ON p . id_pedido = dp . id_pedido

```

9

```

29 GROUP BY u . id_usuario , u . nombre
30 HAVING SUM ( dp . cantidad * dp . precio_unitario ) > 5000.00 ; 31 -- Resultado simulado : 12
clientes encontrados con compras superiores a $5 ,000.
32
33 -- 4. Window functions (RANK , ROW_NUMBER , PARTITION BY) 34 -- Proposito : Rankear los
productos mas caros dentro de cada categoria .
35 SELECT nombre , categoria , precio ,
36 RANK () OVER ( PARTITION BY categoria ORDER BY precio DESC ) as rank_precio
37 FROM producto ;
38 -- Resultado simulado : Retorna el inventario , donde el producto mas caro de " Motor " tiene
rank_precio = 1.
39
40 -- 5. Operaciones de conjuntos (UNION , INTERSECT , EXCEPT ) 41 -- Proposito : Obtener
una lista de correos de usuarios y proveedores ( UNION ) para una campana de marketing
masiva .
42 SELECT email AS correo , 'Cliente' AS tipo_contacto FROM usuarios 43 UNION
44 SELECT email_proveedor AS correo , 'Proveedor' AS tipo_contacto FROM proveedor ;
45 -- Resultado simulado : 10 ,050 correos unicos (10 ,000 clientes + 50 proveedores ) .
46
47 -- 6. Common Table Expressions ( CTEs )
48 -- Proposito : Calcular el ticket promedio mensual de ventas usando una tabla temporal (CTE).
49 WITH VentasMensuales AS (
50 SELECT DATE_TRUNC ( 'month' , p . fecha_pedido ) AS mes , SUM ( pa . monto ) as total_ventas
51 FROM pedido p
52 JOIN pago pa ON p . id_pedido = pa . id_pedido
53 WHERE pa . estado_pago = 'completado '
54 GROUP BY DATE_TRUNC ( 'month' , p . fecha_pedido )
55 )
56 SELECT mes , total_ventas ,
57 total_ventas / ( SELECT COUNT (*) FROM pedido ) AS
    ticket_promedio_general
58 FROM VentasMensuales ;
59 -- Resultado simulado : 3 filas retornadas ( Meses con operaciones y sus respectivos tickets ) .
60
61 -- 7. Consultas con CASE

```

```

62 -- Proposito : Clasificar el stock de productos para alertas de inventario .
63 SELECT nombre , stock ,
64 CASE
65 WHEN stock = 0 THEN 'Agotado ( CRITICO )'
66 WHEN stock < 10 THEN 'Stock Bajo ( REABASTECER )'
67 ELSE 'Stock Saludable '
68 END AS estado_inventario
69 FROM producto ;
70 -- Resultado simulado : Muestra la clasificacion para cada una de las refacciones registradas .
71
72 -- 8. Analisis temporal con fechas
73 -- Proposito : Obtener todos los pedidos realizados en los ultimos 30 dias .
74 SELECT id_pedido , fecha_pedido

```

10

```

75 FROM pedido
76 WHERE fecha_pedido >= CURRENT_DATE - INTERVAL '30 days'; 77 -- Resultado simulado : 230
pedidos realizados en el ultimo mes . 78
79 -- 9. Expresiones regulares o busqueda de texto
80 -- Proposito : Buscar usuarios que usan correos de proveedores gubernamentales o institucionales
mexicanos .
81 SELECT id_usuario , nombre , email
82 FROM usuarios
83 WHERE email ~ '^[a-zA-Z0-9._%+-]+@ (.*\ . gob \ . mx |.*\ . edu \ . mx)$'; 84 -- Resultado simulado : 80
usuarios encontrados con dominios .gob . mx o . edu.mx.
85
86 -- 10. Consulta Mixta ( Window Function + CTE )
87 -- Proposito : Obtener el ultimo pedido de cada cliente para enviar encuesta de satisfaccion .
88 WITH UltimosPedidos AS (
89 SELECT id_pedido , id_usuario , fecha_pedido ,
90 ROW_NUMBER () OVER ( PARTITION BY id_usuario ORDER BY fecha_pedido DESC ) as
num_pedido
91 FROM pedido
92 )
93 SELECT u . nombre , up . fecha_pedido
94 FROM UltimosPedidos up
95 JOIN usuarios u ON up . id_usuario = u . id_usuario
96 WHERE up . num_pedido = 1;
97 -- Resultado simulado : Lista de cada cliente activo con la fecha de su compra mas reciente .
98

```

### 3.2. 3.2 Operaciones INSERT

```

1 -- 1. INSERT con subconsultas
2 -- Proposito : Insertar un pedido automatico ( suscripcion mensual ) para los mejores clientes .
3 INSERT INTO pedido ( id_usuario , fecha_pedido )
4 SELECT id_usuario , CURRENT_TIMESTAMP FROM usuarios WHERE rol = ' cliente_vip ';
5
6 -- 2. INSERT multiple
7 -- Proposito : Registrar multiples metodos de pago de prueba en batch .
8 INSERT INTO pago ( metodo_pago , monto , estado_pago , id_pedido ) VALUES
9 ('tarjeta ' , 1500.00 , 'completado ' , 1) ,
10 ('efectivo ' , 350.50 , 'pendiente ' , 2) ,
11 (' transferencia ' , 5000.00 , 'completado ' , 3) ;
12
13 -- 3. INSERT con valores calculados
14 -- Proposito : Aplicar un recargo de envio (15 %) al costo normal e insertarlo .
15 INSERT INTO PedidoenLinea ( id_pedido_enLinea , costo_envio ) 16 VALUES (4 , ( SELECT SUM (
precio_unitario * cantidad ) * 0.15 FROM detallePedido WHERE id_pedido = 4) ) ;
17
18 -- 4. INSERT con manejo de duplicados ( UPSERT )
19 -- Proposito : Actualizar el numero de telefono del proveedor si el email ya existe en la BD.

```

```

20 INSERT INTO proveedor ( nombre , direccion , email_proveedor , tel_proveedor )
21 VALUES ( 'AutoPartes Express ' , 'Av. Central 123 ' , ' contacto@autopartes .com ' ,
           '5511223344 ' )
22 ON CONFLICT ( email_proveedor )
23 DO UPDATE SET tel_proveedor = EXCLUDED . tel_proveedor ; 24

```

### 3.3. 3.3 Operaciones UPDATE

```

1 -- 1. UPDATE con JOIN ( PostgreSQL usa la clausula FROM para esto ) 2 -- Proposito : Actualizar el
estado de envio si el pago fue marcado como fallido .
3 UPDATE envio e
4 SET estado_envio = 'cancelado '
5 FROM pago p , PedidoenLinea pl
6 WHERE e . id_pedido_enLinea = pl . id_pedido_enLinea 7 AND pl .
id_pedido_enLinea = p . id_pedido
8 AND p . estado_pago = 'fallido ' ;
9
10 -- 2. UPDATE condicional con CASE
11 -- Proposito : Aplicar descuento general de Buen Fin : 20 % a 'Frenos ' , 10 % a 'Motor ' , el resto se
queda igual .
12 UPDATE producto
13 SET precio = CASE
14 WHEN categoria = 'Frenos ' THEN precio * 0.80
15 WHEN categoria = 'Motor ' THEN precio * 0.90
16 ELSE precio
17 END ;
18
19 -- 3. UPDATE masivo
20 -- Proposito : Estandarizar la capitalizacion de la marca 'BOSCH ' . 21 UPDATE producto SET marca =
'Bosch ' WHERE UPPER ( marca ) = 'BOSCH ' ; 22
23 -- 4. UPDATE con subconsultas
24 -- Proposito : Incrementar el stock de un producto si el proveedor principal cambia de direccion .
25 UPDATE producto SET stock = stock + 50
26 WHERE id_proveedor = ( SELECT id_proveedor FROM proveedor WHERE nombre = ' Distribuidor
CDMX Principal ' ) ;
27

```

### 3.4. 3.4 Operaciones DELETE

```

1 -- 1. DELETE con subconsultas
2 -- Proposito : Eliminar detalles de pedidos que pertenecen a pedidos cancelados hace mas de
un ano .
3 DELETE FROM detallePedido
4 WHERE id_pedido IN (
5 SELECT id_pedido FROM pedido WHERE fecha_pedido < CURRENT_DATE - INTERVAL '1 year '
6 ) ;
7
8 -- 2. DELETE con JOIN ( PostgreSQL usa USING )
9 -- Proposito : Eliminar vehiculos de la tabla producto_vehiculo si la marca del vehiculo es descontinuada
.
10 DELETE FROM producto_vehiculo pv
11 USING vehiculo v

```

```

12 WHERE pv . id_vehiculo = v . id_vehiculo AND v . marca = ' MarcaDescontinuada ' ;
13
14 -- 3. Soft delete ( marcado logico )

```

```

15 -- Proposito : Desactivar un usuario ( empleado o cliente ) en lugar de borrarlo fisicamente ( para
    auditoria ).
16 -- Asume la existencia del campo " activo BOOLEAN " ( Agregado en Ejercicio 1)
17 UPDATE usuarios SET activo = FALSE WHERE id_usuario = 15; 18
19 -- 4. Archivado antes de eliminacion
20 -- Proposito : Mover un producto descontinuado a una tabla de historico antes de borrarlo .
21 -- Primero creamos la tabla archivo ( Supuesto )
22 CREATE TABLE IF NOT EXISTS producto_historico ( LIKE producto INCLUDING ALL ) ;
23
24 -- Usamos un CTE con RETURNING para insertar y borrar en un solo paso
25 WITH ProductoBorrado AS (
26 DELETE FROM producto WHERE id_producto = 42 RETURNING * 27 )
28 INSERT INTO producto_historico SELECT * FROM ProductoBorrado ; 29

```

### 3.5. 3.5 Transacciones (Simulaci3n de un proceso de compra completo)

```

1 -- Establecemos nivel de aislamiento para evitar lecturas fantasma durante la venta
2 BEGIN ISOLATION LEVEL SERIALIZABLE ;
3
4 -- Manejo de bloqueos : Bloqueamos la fila del producto para que nadie mas la modifique / compre
5 SELECT stock FROM producto WHERE id_producto = 10 FOR UPDATE ; 6
7 -- Insertamos el pedido
8 INSERT INTO pedido ( id_usuario , fecha_pedido ) VALUES (5 , CURRENT_TIMESTAMP )
    ;
9
10 -- Creamos un punto de guardado por si falla el pago 11 SAVEPOINT
    sp_pedido_creado ;
12
13 -- Intentamos insertar el pago
14 INSERT INTO pago ( metodo_pago , monto , estado_pago , id_pedido ) 15 VALUES ('tarjeta ',
    2500.00 , 'fallido ', curval ( ' pedido_id_pedido_seq ' ) ) ;
16
17 -- Control de errores y rollback : Como el pago fallo , regresamos al savepoint
18 ROLLBACK TO sp_pedido_creado ;
19
20 -- Intentamos con otro metodo de pago
21 INSERT INTO pago ( metodo_pago , monto , estado_pago , id_pedido ) 22 VALUES ('efectivo ',
    2500.00 , 'pendiente ', curval ( ' pedido_id_pedido_seq ' ) ) ;
23
24 -- Si el pago paso a pendiente , descontamos stock 25 UPDATE producto SET stock = stock - 1
    WHERE id_producto = 10; 26

```

13

```

27 -- Si todo esta bien hasta aqui , guardamos permanentemente en BD 28 COMMIT ;
29

```



=====

===

CUESTIONARIO - INTEGRIDAD REFERENCIAL Y CONSULTAS SQL

Base de Datos: universidad\_avanzada

Fecha: 2025-10-28

=====

===

INSTRUCCIONES:

- Lee cada pregunta cuidadosamente antes de responder
- Escribe tu respuesta en la línea "R:"
- Para preguntas SQL incluye la consulta en "SQL:"
- Consulta los archivos DDL para verificar políticas de integridad

=====

===

1. ¿Qué política ON DELETE tiene la tabla 'grupos' hacia 'asignaturas'?

R:

Explicación: Este sirve para que si borrar todas las tablas hijas que tiene, entonces si se elimina Asignatura se eliminaria, grupos

2. Si eliminas la asignatura 'CS301', ¿qué sucede con sus grupos? R: Pues se eliminan todos sus grupos y todo lo demas que tenga como hijas SQL para intentar:

3. ¿Qué política ON DELETE tiene la tabla 'profesores' hacia 'departamentos'?

R:



Explicación: SI se borra la tabla padre el valor de el hijo se pone directamente NULL, en este caso si se borra un registro de la tabla profesores el departamento se pone NULL

4. Si eliminas el departamento con id=1, ¿qué pasa con sus profesores? R:

Resultado esperado: Basicamente porque tiene un RESTRICT para que las asignaturas no queden huérfanas

5. ¿Por qué NO puedes eliminar la asignatura 'CS101'?

R:

Política que lo impide: El RESTRICT es un requisito para cursar otras materias

6. ¿Qué política ON DELETE tiene la tabla 'inscripciones' hacia 'estudiantes'?

R:

Efecto al eliminar estudiante: Se eliminarían las llaves foráneas de las tablas inscripciones, tutorías y evaluaciones\_docentes

7. Si eliminas el estudiante '2022-001', ¿qué sucede con sus inscripciones?

R:

SQL: DELETE FROM estudiantes WHERE matricula = 2022-001'

8. ¿Qué política ON DELETE tiene la tabla 'tutorías' hacia 'profesores'? R:

¿Qué pasa con las tutorías si eliminas un profesor?:

9. Si eliminas el proyecto con id\_proyecto=3, ¿qué sucede con sus colaboradores?

R:

Política aplicada:

10. ¿Qué política ON DELETE tiene 'autores\_publicacion' hacia 'publicaciones'?

R:

Efecto al eliminar una publicación:

11. ¿Qué política ON UPDATE tienen TODAS las foreign keys en esta BD? R:

Beneficio de esta política:

12. Si actualizas el DNI del profesor 'P001' a 'P999', ¿qué tablas se actualizan?

R:

SQL para verificar:

13. Si cambias el código de 'CS101' a 'CS001', ¿se actualizan automáticamente los grupos?

R:

Política que lo permite:

14. Si actualizas id\_departamento de 1 a 99, ¿qué tablas se ven afectadas? R:

Tablas:

15. ¿Por qué es importante ON UPDATE CASCADE en llaves foráneas? R:  
Ejemplo real:

16. Diferencia entre CASCADE y RESTRICT en ON DELETE  
R:  
Ejemplo de cada uno en esta BD:

17. Diferencia entre SET NULL y CASCADE en ON DELETE  
R:  
¿Cuándo usar cada uno?:

18. ¿Por qué 'prerequisitos' usa RESTRICT en lugar de CASCADE? R:  
Justificación académica:

19. ¿Por qué 'estudiantes.tutor\_dni' usa SET NULL en lugar de RESTRICT? R:  
Caso de uso:

20. ¿Cuándo es apropiado usar RESTRICT en lugar de CASCADE? R:  
Criterios de decisión:

21. Lista el nombre completo de todos los profesores con su departamento R:  
SQL:

22. ¿Cuántos estudiantes están inscritos en asignaturas del departamento 'Ciencias de la Computación'?  
R:  
SQL:

23. Muestra todas las asignaturas con sus prerequisitos (nombre de ambas) R:  
SQL:

24. ¿Cuántos profesores de categoría 'TIT' imparten grupos en semestre 2024- 1?  
R:  
SQL:

25. ¿Cuántos estudiantes tienen como tutor a un profesor del departamento 'Matemáticas'?  
R:  
SQL:

26. ¿Cuál es el nombre del profesor con el salario más alto? R:  
SQL:

27. ¿Cuántos profesores ganan más que el salario promedio? R:  
SQL:

28. ¿Cuál es el nombre del estudiante con el promedio más alto? R:  
SQL:

29. ¿Cuántas asignaturas tienen más créditos totales (teoría+práctica) que el promedio?  
R:  
SQL:

30. ¿Qué departamento tiene el mayor presupuesto? (nombre) R:  
SQL:

31. ¿Qué departamento tiene más profesores? (nombre y cantidad) R:  
SQL:

32. ¿Qué profesor imparte más grupos en semestre 2024-1? (nombre completo) R:

SQL:

33. ¿Cuántos estudiantes tiene cada tutor? (solo tutores con más de 2 estudiantes)

R:

SQL:

34. ¿Cuál es el promedio de calificaciones finales de estudiantes aprobados? R:

SQL:

35. ¿Cuántas asignaturas hay por cada nivel? (Básico, Intermedio, Avanzado, Especialización)

R:

SQL:

36. ¿Cuántos grupos están al límite o sobrecupo (inscritos  $\geq$  cupo)? R:

SQL:

37. ¿Cuántas asignaturas NO tienen prerequisites?

R:

SQL:

38. ¿Cuántos profesores NO imparten grupos en el semestre 2024-1? R:

SQL:

39. ¿Cuántos estudiantes tienen promedio superior al promedio general? R:

SQL:

40. ¿Cuál es la asignatura más popular (más inscripciones) en semestre 2024- 1?

R:

SQL:

=====  
===  
FIN DEL CUESTIONARIO  
=====  
===